

数论中的同余代数和数论函数是数论在 OI 中的主要应用，本篇笔记总结了同余代数相关知识点。

尽管近年来的重大考试当中少有涉及同余代数的题目，但熟练掌握相关的各种算法和它们的推导过程对我们的思维水平有很大的锻炼。

数论和抽象代数的联系是非常密切的，以至于同余代数几乎完全属于抽象代数的范畴。如果能够从抽象代数（尤其是循环群）的角度自顶向下地学习同余代数（在 OI 中），那么它将非常简单且容易理解。例如：

- 循环群在同余代数中的最直接应用就是模 p 下的加法（见 1.10 小节）。
- 中国剩余定理的另一种形式 $\mathbb{Z}/n\mathbb{Z} \cong \mathbb{Z}/n_1\mathbb{Z} \times \mathbb{Z}/n_2\mathbb{Z} \times \cdots \times \mathbb{Z}/n_k\mathbb{Z}$ （见 3.3 小节）。
- 考察 $(\mathbb{Z}/n\mathbb{Z})^\times$ 的结构性质给出了原根的存在性定理和在模意义下开 N 次根的算法（见 6.3 小节）。

打星号的小节（不含例题）需要一些抽象代数的前置知识，见 OI 的群论部分或抽象代数课程笔记。链接附在了参考资料部分。

定义与记号

Abstractness is the price of generality.

为了更好地理解同余理论相关算法，读者需熟知基本概念，如素数的定义，算数基本定理，同余符号及其含义，最大公约数等，并掌握基本算法如快速幂，辗转相除法求最大公约数（Euclid 算法）等。

说明

在模 1 下讨论整数同余毫无意义，下文所有同余式的模数默认 $p > 1$ ，有 $a \cdot 1 \equiv a \pmod{p}$ 。

定义

- 整除：若非零整数 a 是整数 b 的因数，则称 a **整除** b 或 b 被 a 整除，记作 $a \mid b$ 。如 $2 \mid 4$ ， $822 \nmid 1064$ 。
- 同余：若整数 a, b 模正整数 p 的余数相等，则称 a, b **在模 p 下同余**，记作 $a \equiv b \pmod{p}$ 。如 $15 \equiv 57 \pmod{7}$ ， $-1 \equiv 7 \pmod{8}$ 。
- 最大公约数：整数 a 和 b 的 **最大公约数** 为最大的整数 d 满足 d 整除 a 和 b ，记作 $\gcd(a, b)$ 。如 $\gcd(4, 6) = 2$ ， $\gcd(0, n) = n$ 。
- 互质：若整数 a, b 满足 $\gcd(a, b) = 1$ ，则称 a, b **互质**，记作 $a \perp b$ 。一般 a, b 均为非负整数。如 $3 \perp 8$ ， $1 \perp n$ ， $4 \nperp 6$ 。
- 剩余类：模 p 同余的所有数构成的等价类被称为模 p 的 **剩余类**。模 p 下它们等价。
- 完全剩余系： $\{0, 1, \cdots, p-1\}$ 构成模 p 的 **完全剩余系**，记为 Z_p 。

- 简化剩余系：所有小于 n 且与 n 互质的正整数构成模 n 的 **简化剩余系**，又称 **既约剩余系** 或 **缩系**，记为 $(\mathbb{Z}/n\mathbb{Z})^*$ 。这个集合的大小为 $\varphi(n)$ 。

注意区分因数和因子：因数表示一个数的约数，因子表示乘积的一项。

记号

- 素数集：记 $\mathbb{P}$ 表示素数集。 $\mathbb{P} = \{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, \dots\}$ 。
- 质因数次数： n 所含质因数 p 的次数记作 $\nu_p(n)$ 。 $p^{\nu_p(n)} \mid n$ 且 $p^{\nu_p(n)+1} \nmid n$ 。
- 各位数字和： n 在 p 进制下的各位数字之和记作 $s_p(n)$ 。

基本知识（非常重要）

- 给定 a, b ，使得 $a \mid bx$ 的最小的 x 为 $\frac{a}{\gcd(a,b)}$ ，且所有这样的 x 恰为 $\frac{a}{\gcd(a,b)}$ 的倍数。对每个质因数单独分析即得。特别地，若 $bx \equiv 0 \pmod{a}$ 且 $a \perp b$ ，则 $x \equiv 0 \pmod{a}$ 。
- 对 $d \mid m$ ，在 $1 \sim m$ 中使得 $\gcd(m, x) = d$ 的 x 的个数等于在 $1 \sim \frac{m}{d}$ 中使得 $\frac{m}{d} \perp x$ 的 x 的个数，即 $\varphi(\frac{m}{d})$ 。因为 $\gcd(m, x) = d$ 等价于 $\gcd(\frac{m}{d}, \frac{x}{d}) = 1$ 。

1. 基础知识

OI 中的同余理论主要是 **在模意义下解方程**。

1.1 同余的性质

了解同余的性质是学习同余数论的基础。同余符号的根本含义： $a \equiv b \pmod{p}$ 等价于 $a = b + kp$ ($k \in \mathbb{Z}$)，前者是后者的简要写法。

在同余式中，加法和乘法的交换律与结合律以及乘法分配律依然成立。以下两条性质是进行模意义下计算的依据。

- 两整数相加（减）再对 p 取模等于两个数先对 p 取模再相加（减），再对 p 取模。
- 两整数相乘再对 p 取模等于两个数先对 p 取模再相乘，再对 p 取模。

证明

设 $a = q_1p + r_1$ ， $b = q_2p + r_2$ ，其中所有数都是整数且 $0 \leq r_1, r_2 < p$ ，即 $r_1 = a \bmod p$ ， $r_2 = b \bmod p$ 。因为 $kp \bmod p = 0$ ，所以

$$(a + b) \bmod p = (q_1p + r_1 + q_2p + r_2) \bmod p = ((a \bmod p) + (b \bmod p)) \bmod p$$

乘法同理。□

和等式一样，同余式的基本性质：若 $a \equiv b \pmod{p}$ ，则 $a + c \equiv b + c \pmod{p}$ ， $ac \equiv bc \pmod{p}$ 。

需要注意，即使 $ac \equiv bc \pmod{p}$ ，也不一定有 $a \equiv b \pmod{p}$ 。 $c \perp p$ 是命题成立（ c 有乘法逆元）的充要条件。关于同余式的除法，见 1.3 小节。

1.2 Fermat 小定理

要求模数 p 为质数。

引理

当 p 是质数时，其因数只有 1 和 p 。因此，若两个数相乘是 p 的倍数，则其中至少有一个是 p 的倍数。

这个引理很重要。

当 a 不是 p 的倍数时，不存在 $x \neq y$ 且 $1 \leq x, y < p$ 满足 $xa \equiv ya \pmod{p}$ ，否则 $x - y$ 是 p 的倍数，与 $1 \leq x, y < p$ 的限制矛盾。

考虑 $1 \sim p - 1$ 所有数，它们乘以 a 之后在模 p 下互不相同且不为 0，所以仍得 $1 \sim p - 1$ 所有数。于是

$$\prod_{i=1}^{p-1} i \equiv \prod_{i=1}^{p-1} ai \equiv a^{p-1} \prod_{i=1}^{p-1} i \pmod{p} \implies (a^{p-1} - 1) \prod_{i=1}^{p-1} i \equiv 0 \pmod{p}$$

因为 $\prod_{i=1}^{p-1} i$ 不是 p 的倍数，所以

$$a^{p-1} \equiv 1 \pmod{p}$$

该结论称为 **Fermat 小定理**。根据推导过程，它适用于 p 是质数且 a 不是 p 的倍数的情形。

1.3 乘法逆元

定义

为了支持同余式的除法运算，需要引入乘法逆元的概念：对整数 a ，若 $ax \equiv 1 \pmod{p}$ ，则称 x 为 a 在模 p 下的 **乘法逆元**，记作 $(a^{-1})_p$ 。模 p 同余式中出现乘法逆元，除非在符号的右下角标记模数，否则认为逆元在模 p 下，可直接写 a^{-1} 。

乘法逆元不一定存在，也不一定唯一。在实际运算中，只需要任意一个。

考虑同余式 $ax \equiv b \pmod{p}$ ，若希望将同余式两侧同时除以 a ，则只需乘以 a 的乘法逆元：

$$aa^{-1}x \equiv ba^{-1} \pmod{p} \implies x \equiv ba^{-1} \pmod{p}$$

求法

根据 Fermat 小定理，当 p 为质数时， $a \cdot a^{p-2} \equiv 1 \pmod{p}$ ，即

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

据此快速幂求出一个数在模质数下的乘法逆元。

当 p 不是质数时， a 存在乘法逆元当且仅当 $a \perp p$ 。我们将在第二章展开讨论。

以下给出几个模 **质数** p 下求乘法逆元的常见技巧。

前缀逆元

增量法，设 $1 \sim i-1$ 的逆元已知。

设 $p = ki + r$ ($0 \leq r < i$)，即 k 和 r 分别是 p 对 i 做带余除法的商和余数。 $ki + r \equiv 0 \pmod{p}$ ，两边同时除以 ir ，得

$$i^{-1} \equiv -kr^{-1} \equiv -\left\lfloor \frac{p}{i} \right\rfloor (p \bmod i)^{-1}$$

时间复杂度线性。[模板题](#)。

离线 $\mathcal{O}(1)$ 逆元

求任意 n 个数 a_i ($1 \leq a_i < p$) 的逆元。

设 s 为 a 的前缀积。通过 Fermat 小定理算出 s_n^{-1} 后，计算 $s_i^{-1} \equiv s_{i+1}^{-1} \times a_{i+1}$ 得到前缀积的逆元，则 $a_i^{-1} \equiv s_{i-1} \times s_i^{-1}$ 。

时间复杂度 $\mathcal{O}(n + \log p)$ 。[模板题](#)。

在线 $\mathcal{O}(1)$ 逆元

首先 $\mathcal{O}(k)$ 预处理一段前缀 $[1, k]$ 的逆元。对询问的 x ，找到 u 使得 $xu \bmod p \leq k$ ，则 $x^{-1} = u(xu \bmod p)^{-1}$ 。

不能对每个 x 都预处理对应的 u ，所以我们只能对一些 x 预处理。考虑将 $1 \sim p-1$ 分成若干组，每组的 u 可共用或根据组代表元的 u 快速计算。

最简单的分组方式是整除。设阈值 B , $x = qB + r$, 则 $xu \equiv qBu + ru \pmod{p}$, 考虑让 ru 较小且 $qBu \pmod{p}$ 较小。设 u 的阈值为 B_u , 则 $ru < BB_u$ 。令 $qBu \pmod{p}$ 和 ru 差不多, 则时间复杂度至少为 $\Omega(BB_u + \frac{p}{B})$ 。取 $B = B_u = p^{\frac{1}{3}}$ 较优秀。

目标: 对每个 q 求出 $0 < u \leq p^{\frac{1}{3}}$ 使得 $qBu \pmod{p} = v$, 满足 $|v| \leq p^{\frac{2}{3}}$ 。抽屉原理保证解存在: 对 $qBu_1 - v_1 \equiv qBu_2 - v_2 \pmod{p}$, $(u_1 - u_2, v_1 - v_2)$ 即一组解。

枚举每个 u , 考虑 q 从 0 增加到 $\frac{p}{B}$, 每次增加 1 则 v 增加 $Bu \leq p^{\frac{2}{3}}$ 。若当前 q 对应的 v 不合法, 因为合法 v 的区间长度大于 $p^{\frac{2}{3}}$, 所以可立即找到下个合法 q 。

对每个 u , 共跳过 $\mathcal{O}(\frac{p^{\frac{2}{3}}Bu}{p})$ 次 (qBu 的最大值除以 p), 每次跳过带来 $\mathcal{O}(\frac{p^{\frac{2}{3}}}{Bu})$ 个合法 q (合法区间长度除以步长), 共有 $\mathcal{O}(p^{\frac{1}{3}})$ 个合法的 q 。

复杂度 $\mathcal{O}(n + p^{\frac{2}{3}})$ 。模板题。

1.4 Lagrange 定理

我们知道代数学基本定理: n 次多项式在复数域 $\mathbb{C}$ 上有且仅有 n 个可重根。那么, 模意义下的多项式是否仍然满足该性质呢?

Lagrange 定理: 对于 $\mathbb{Z}_p$ ($p \in \mathbb{P}$) 上的整系数多项式 $f(x) = \sum_{i=0}^n a_i x^i$, 其中 $p \nmid a_n$, 则 $f(x) \equiv 0 \pmod{p}$ 在模 p 下 **至多** 有 n 个不同解。

证明

考虑数学归纳法。当 $n = 0$ 时定理成立, 因为 $a_0 \equiv 0 \pmod{p}$ 无解 (注意 $p \nmid a_0$)。

当 $n > 0$ 时, 假设方程有 $n + 1$ 个不同解 $x_0, x_1, \dots, x_n$ 。因为 $f(x_0) \equiv 0 \pmod{p}$, 所以 $f(x) \equiv (x - x_0)g(x) \pmod{p}$, 其中 $g(x)$ 是不超过 $n - 1$ 次的多项式。

因为 $x_i \neq x_0$ ($1 \leq i \leq n$) 且 $(x_i - x_0)g(x_i) \equiv 0 \pmod{p}$, 故 $x_1, x_2, \dots, x_n$ 均为 $g(x)$ 的根, 矛盾。□

解的数量并非恰为 n , 而是不超过 n , 例如 $x^2 - 2 \equiv 0 \pmod{3}$ 无解, 这是 Lagrange 定理和代数学基本定理的区别之一。所以 $f(x)$ 在模 p 下并不一定可以被分解为 n 个一次式的乘积。

- 注意: p 不是质数时结论不成立, 如 $x^2 \equiv 1 \pmod{8}$ 有解 $x = 1, 3, 5, 7$ 。

1.5 Wilson 定理

一般形式

乘法逆元是相互的。若 a 是 x 的乘法逆元，则 x 也是 a 的乘法逆元。这启发我们思考：当 p 是质数时， $1 \sim p-1$ 所有数能否两两配对互为逆元？

对 $p \in \mathbb{P}$ 且 $p > 2$ ，考虑 $x^2 \equiv 1 \pmod{p}$ 有解 $x = \pm 1$ (Lagrange 定理保证不会有其它解存在)，于是 $2 \sim p-2$ 两两配对互为逆元，即 $\prod_{i=2}^{p-2} i \equiv 1 \pmod{p}$ 。因此 $(p-1)! \equiv 1 \cdot (p-1) \equiv -1 \pmod{p}$ 。对 $p=2$ 成立。

考虑 p 不是质数的情况：

- 当 $p = q^2$ 时，考虑 q 和 $2q$ 。它们相乘后模 p 等于 0。因此，若 $2q < p$ ，即 p 为大于 4 的完全平方数时， $(p-1)! \equiv 0 \pmod{p}$ ；
- 当 $p = 4$ 时， $3! \equiv 2 \pmod{4}$ ；
- 当 p 为大于 4 的非完全平方数时，令 q 为 p 的最小质因数，则 $q \neq \frac{p}{q}$ ，所以 $(p-1)! \equiv 0 \pmod{p}$ 。

综上，我们得到 **Wilson 定理**： $(p-1)! \equiv -1 \pmod{p}$ 当且仅当 p 是质数。

扩展形式

尝试将 Wilson 定理扩展至素数幂的情形。

考虑 p^k 以内所有与 p 互质的数的乘积模 p^k ，记为 $(p^k!)_p$ 。

仍然考虑求解 $x^2 \equiv 1 \pmod{p^k}$ ，求出所有逆元为本身的数，并将不为 x 的其它所有数与其逆元两两配对。因为我们只考虑与 p 互质的数，所以逆元存在。因此只需考虑所有 x 的乘积。

因式分解得 $(x+1)(x-1) \equiv 0 \pmod{p^k}$ 。

当 $p > 2$ 时， $x+1$ 和 $x-1$ 不可能均是 p 的倍数，必然有 $x+1 \equiv 0 \pmod{p^k}$ 或 $x-1 \equiv 0 \pmod{p^k}$ 。故仍有 $(p^k!)_p \equiv -1 \pmod{p^k}$ 。

当 $p=2$ 时，有平凡解 ± 1 。除此以外， $x+1$ 和 $x-1$ 可能同时为 2 的倍数。但它们不可能同时为 4 的倍数。若 $x+1$ 和 $x-1$ 同时为 2 的倍数，那么将 $(x+1)(x-1) \equiv 0 \pmod{2^k}$ 两侧除掉不能被 4 整除的数（模数除以 2，因为被除掉的数恰好贡献一个质因数 2），得到 $x \pm 1 \equiv 0 \pmod{2^{k-1}}$ 。当 $p=2$ 时，方程有四个解 ± 1 和 $2^{k-1} \pm 1$ 。

注意 $k=1$ 时四个解均重合，此时 $1! \equiv -1 \pmod{2}$ 。 $k=2$ 时两对解重合，此时 $1 \times 3 \equiv -1 \pmod{4}$ 。否则 $1 \times (-1) \times (2^{k-1} + 1) \times (2^{k-1} - 1) \equiv 1 \pmod{2^k}$ 。

综上，得到 Wilson 定理的扩展形式

$$(p^k!)_p \equiv \begin{cases} 1 & p=2 \wedge k \geq 3 \\ -1 & \text{otherwise} \end{cases} \pmod{p^k}$$

在 exLucas 中用到了该结论。

1.6 Kummer 定理

阶乘的素数幂次

给定正整数 n 和质数 p ，求 $\nu_p(n!)$ 。

提出 $1 \sim n$ 当中所有 p 的倍数，将它们全部除以 p 。这一步消掉的质因数个数为 $\lfloor \frac{n}{p} \rfloor$ 。上一步操作后，得到 $1 \sim \lfloor \frac{n}{p} \rfloor$ 。将所有 p 的倍数提出来，然后全部除以 p 。这一步消掉的质因数个数为 $\lfloor \frac{n}{p^2} \rfloor$ 。以此类推，直到 $\lfloor \frac{n}{p^k} \rfloor < p$ 。此时已经没有 p 的倍数了。有如下结论：

$$\nu_p(n!) = \sum_{i=1}^{\lfloor \log_p n \rfloor} \left\lfloor \frac{n}{p^i} \right\rfloor$$

该结论由 Legendre 在 1808 年提出。

考虑换一种方法求和。尝试对 n 在 p 进制下的每一位，求出它对每次提出因数个数的贡献之和。令 $c = \lfloor \log_p n \rfloor$ ，设 $n = \sum_{i=0}^c a_i p^i$ ($0 \leq a_i < p$)，则

$$\begin{aligned} \nu_p(n!) &= \sum_{i=0}^c \sum_{j=1}^c \left\lfloor \frac{a_i p^i}{p^j} \right\rfloor \\ &= \sum_{i=0}^c a_i \sum_{j=0}^{i-1} p^j \\ &= \sum_{i=0}^c a_i \frac{p^i - 1}{p - 1} \\ &= \frac{(\sum_{i=0}^c a_i p^i) - (\sum_{i=0}^c a_i)}{p - 1} \\ &= \frac{n - s_p(n)}{p - 1} \end{aligned}$$

其中用到了等比数列求和公式 $\sum_{i=0}^k p^i = \frac{p^{k+1} - 1}{p - 1}$ 。

通过上述推导，得到 $\nu_p(n!)$ 的另一种表示方法： n 减去 n 在 p 进制下的各位数字和，再除以 $p - 1$ 。特别地，当 $p = 2$ 时， $\nu_2(n!) = n - \text{popcount}(n)$ 。

组合数的素数幂次

根据组合数公式 $\binom{n}{m} = \frac{n!}{m!(n-m)!}$ ，有

$$\begin{aligned} & \nu_p \left(\binom{n}{m} \right) \\ &= \frac{n - s_p(n) - (m - s_p(m)) - (n - m - s_p(n - m))}{p - 1} \\ &= \frac{s_p(m) + s_p(n - m) - s_p(n)}{p - 1} \end{aligned}$$

考虑 $n - m$ 和 m 在 p 进制下相加得到 n 的过程。每产生一次进位，各位数字之和就会减少 $p - 1$ ，因此 p 在 $\binom{n}{m}$ 中的幂次等于 p 进制下 $n - m$ 加 m 需要进位的次数。

该结论称为 **Kummer 定理**。

1.7 步长与子环

结论

在长为 n 的环上每一步走 k 条边，形成 $d = \gcd(n, k)$ 个子环，每个环的环长为 $\frac{n}{d}$ 。

按顺序给环上的每个点标号 $0, 1, 2, \dots, n - 1$ 。从 r 开始每一步走 k 条边，经过的点的编号与 r 在模 d 下同余，因为 n, k 均为 d 的倍数， r 加上 k 对 n 取模不改变 $r \bmod d$ 。

引理

当 $n \perp k$ 时，从环上任意一点出发，能够走遍所有点。

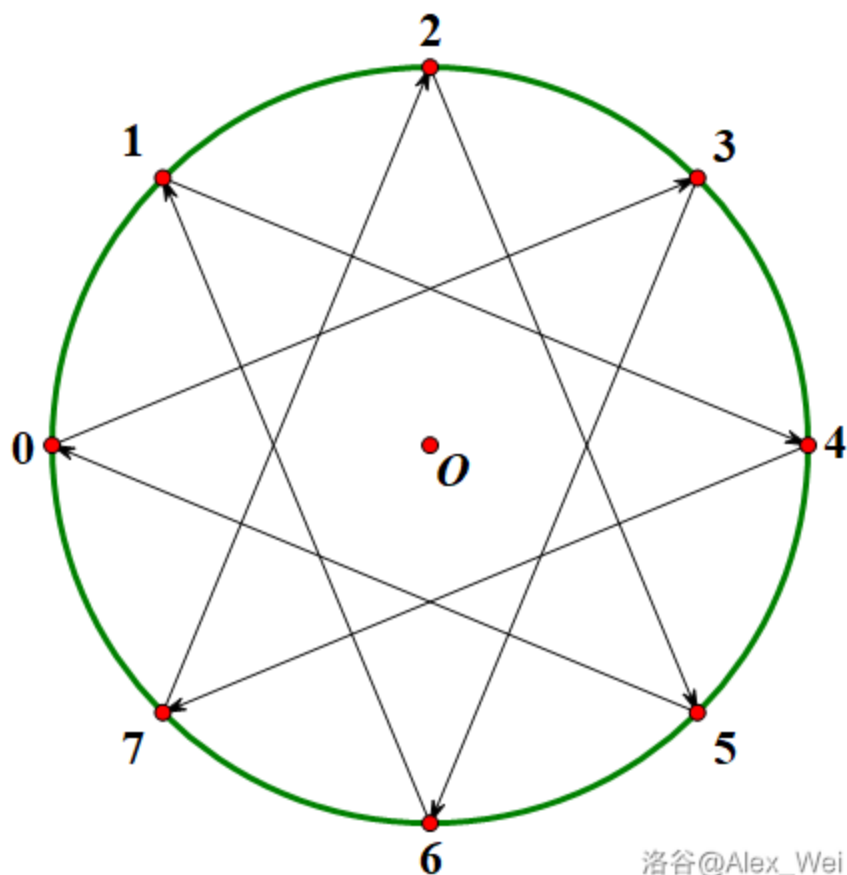
证明

即证 $(r + ck) \bmod n$ 对 $0 \leq c < n$ 互不相同。假设存在 $0 \leq i < j < n$ 有 $r + ik \equiv r + jk \pmod{n}$ ，移项得 $(j - i)k \equiv 0 \pmod{n}$ 。因为 $k \perp n$ ，所以 $n \mid j - i$ ，这与 $0 \leq i, j < n$ 矛盾。□

说明

这个引理本质上和最开始的基本知识等价。如果跳 x 步回到原来的位置，那么 $n \mid kx$ ，推出 x 是 $\frac{n}{\gcd(n, k)}$ 的倍数。但 $n \perp k$ ，所以 x 无论如何都是 n 的倍数。

即当 $k \perp n$ 时， $ck \bmod n$ 取遍 $0 \sim n - 1$ ，这是 Fermat 小定理的引理的推广，可以进一步推出 Euler 定理。

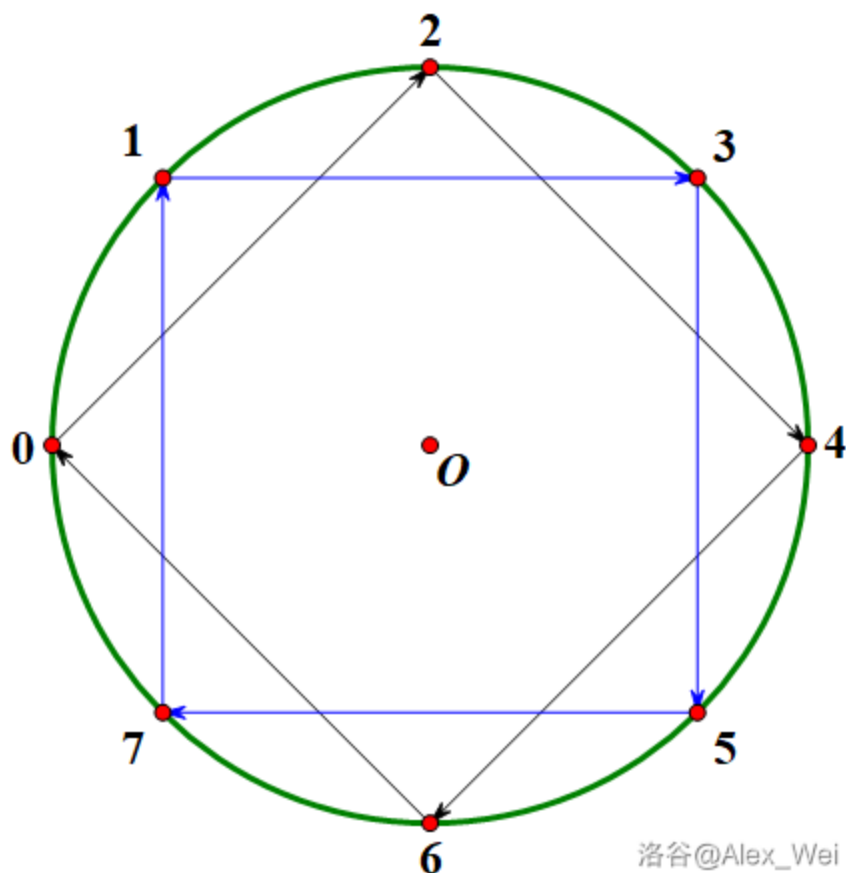


洛谷@Alex_Wei

不妨设 $0 \leq r < d$ 。考虑 $0 \sim n-1$ 当中所有模 d 余 r 的数 $r, r+d, \dots, r+(\frac{n}{d}-1)d$ ，将它们重标号为 $0, 1, \dots, \frac{n}{d}-1$ ，形成长度为 $\frac{n}{d}$ 的子环。

因为 d 是 n, k 的最大公约数，所以 $\frac{n}{d} \perp \frac{k}{d}$ 。根据引理，在这个长为 $\frac{n}{d}$ 的子环上每一步走 $\frac{k}{d}$ 条边（相当于在原环上走 k 条边），能够走遍子环上的每一个点。

于是子环长为 $\frac{n}{d}$ 即 $\frac{n}{\gcd(n,k)}$ 。每个模 d 的剩余系形成一个长为 $\frac{n}{d}$ 的子环，子环个数为 $d = \gcd(n, k)$ 。



洛谷@Alex_Wei

例题：[P5582](#)，[P5887](#)，[P6187](#)。

1.8 Euler 定理

前置知识：Euler 函数（见数论 II）。

一般形式

回顾 Fermat 小定理的证明过程，我们发现 p 是质数的条件只是为了保证 $ax \bmod p$ 互不相同。由 1.7 小节的引理，这个条件可以仅由 $a \perp p$ 保证。同时我们知道，两个模 p 互质的数相乘后模 p 仍与 p 互质。因此，我们尝试将费马小定理扩展至更一般的情况。

设 K_p 为 p 的简化剩余系。对于任意 $x, y \in K_p$ ，若 $x \neq y$ ，则 ax 和 ay 模 p 的余数不相等且仍与 p 互质。因此，在模 p 下， K_p 中的每个数乘以 a 后仍与 K_p 相等。故 $\prod_{x \in K_p} x \equiv \prod_{x \in K_p} ax \pmod{p}$ 。

等式两边同时除以 $\prod_{x \in K_p} x$ 得到 **Euler 定理**

$$a^{\varphi(p)} \equiv 1 \pmod{p}$$

在计算与模数互质的某个数的幂时，指数可以对模数的 Euler 函数取模。可以用来化简公式或减小常

数，前提是模数是定值或其 Euler 函数容易计算。

当 p 是质数时， $\varphi(p) = p - 1$ ，与 Fermat 小定理一致。

扩展形式

当 $\gcd(a, p) \neq 1$ 时，不断乘以 a 会让 a^i 和 p 的 $\gcd$ 不断增加，直到某个特定的 a^r 之后 $\gcd$ 不再变化，因为此时 $\gcd(a^r, p)$ 的每个质因数受到的限制都是 p 提供的。将 $\gcd$ 除掉之后得到 Euler 定理的条件，于是有 **exEuler 定理**

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(p)} & \gcd(a, p) = 1 \\ a^b & \gcd(a, p) \neq 1, b < \varphi(p) \\ a^{b \bmod \varphi(p) + \varphi(p)} & \gcd(a, p) \neq 1, b \geq \varphi(p) \end{cases} \pmod{p}$$

证明

设 $d = \gcd(a^r, p)$ 。在模 p 下考虑 $a^r, a^{r+1}, \dots$ ，它们均为 d 的倍数。因为 $a \perp \frac{p}{d}$ ，所以它们除以 d 之后会取遍所有与 $\frac{p}{d}$ 互质的数。根据 Euler 定理， $a^k \bmod \frac{p}{d}$ 有循环节 $\varphi(\frac{p}{d})$ ，因此 $(a^{r+k} \bmod \frac{p}{d}) \times d$ 即 $a^{r+k} \bmod p$ 也有长度为 $\varphi(\frac{p}{d})$ 的循环节。而 $\varphi(\frac{p}{d}) \mid \varphi(p)$ ，所以从 a^r 开始，对 $k \geq 0$ ， $a^{r+k} \bmod p$ 有长度为 $\varphi(p)$ 的循环节。

还需证明 $r \leq \varphi(p)$ 。感性理解就是如果 $\gcd$ 变化，一定会乘上一个因子，所以 r 是 $\mathcal{O}(\log p)$ 级别的，会比 $\varphi(p)$ 小。具体怎么证明呢？我们发现 r 就是 a 的每个质因数幂次 $q_i^{k_i}$ 对应的 r_i 的最大值，所以只需对 a 是质数幂次 q^k 的情况证明。设 p 含 c 个质因数 q ，则 $r = \lceil \frac{c}{k} \rceil$ 。因此，显然地，我们只需对 a 是质数 q 的情况证明，因为此时 r 最大。

设 $p = q^c p'$ ，则 $r = c$ 。由 Euler 函数的性质， $\varphi(p) \geq \varphi(q^c) = q^{c-1}(q-1) \geq c$ 。□

1.9 同余式的除法

对于 $a \equiv b \pmod{p}$ ，一种除法是将 a, b 同时除以 $d = \gcd(a, p)$ 以化简同余式（如果 b 无法除尽，则同余式不可能成立），此时 p 也应当除以 $\gcd(a, p)$ 。另一种除法是在 $a \perp p$ 时，将同余式两侧同时除以 a ，通过乘以 a 的乘法逆元实现，此时 p 保持不变。

读者需要区分这两种对同余式的基本变形方法。同余式的本质为存在 k 使得 $a + kp = b$ ，第一种除法相当于将等式两侧同时除以 d 得到 $\frac{a}{d} + k\frac{p}{d} = \frac{b}{d}$ ，因为等式左侧是整数，所以右侧也必须是整数，即 $d \mid b$ 。第二种除法相当于 $ax + kpx = bx$ ，其中 $ax \equiv 1 \pmod{p}$ ，于是存在 k' 使得 $1 + k'p = bx \bmod p$ ，即 $1 \equiv bx \pmod{p}$ ，这是对同余式的简单变形。

*1.10 循环群和直积

循环群 是只由一个元素生成的群 $G = \langle x \rangle = \{x^n \mid n \in \mathbb{Z}\}$ 。大小相同的有限循环群是同构的。大小为 p 的循环群 (p 阶循环群) 记为 Z_p , 则 $x^p = 1$ (这里的 1 是单位元)。

对 Z_p 的直观理解是一个 p 个点的环 $0 \rightarrow 1 \rightarrow \cdots \rightarrow p-1 \rightarrow 0$, 编号为 i 的点对应群内的元素 x^i , 于是元素之间的乘法运算 (定义一个群时的二元运算通常看成元素之间的乘法运算, 但注意这并不是整数乘法, 尽管它们的运算规则几乎一致) 可以看成在环上移动。乘以 x^i 相当于在环上跳 i 步。为什么要想象成一个环呢? 因为这样就可以用 1.7 小节的结论得到一些不那么显然的结论。

Z_p 在同余代数中的最直接应用就是模 p 下的代数加法。因为 $x^p = 1$, 所以 Z_p 上的运算关于 x 的指数是在模 p 下的。这也是为什么模 p 的完全剩余系记作 $Z_p = \{0, 1, \cdots, p-1\}$: 用 i 代替 x^i , 元素的乘法定义为模 p 下的加法。

特别地, 当同时需要模 p 下的加法和乘法时, 会把这个代数结构写成两个环 (环同时定义了乘法和加法, 其中加法构成交换群) 的商 $\mathbb{Z}/p\mathbb{Z}$ 。 $(\mathbb{Z}/p\mathbb{Z})^\times$ 表示所有和 p 互质的数模 p 构成的乘法群。

定义两个群 G, H 的 **直积** 为 $G \times H = \{(x, y) \mid x \in G, y \in H\}$, 其上的二元运算定义为 $(x_1, y_1)(x_2, y_2) = (x_1x_2, y_1y_2)$ 。简单地说, 就是用一个二元组把 G 和 H 绑起来, 但是 G 和 H 在 $G \times H$ 上的运算仍保持一定的独立性。

2. 二元线性不定方程

扩展 Euclid 算法 (exgcd) 用于求解形如 $ax + by = c$ 的 **二元线性不定方程**, 其中 a, b, c, x, y 都是整数, x, y 是未知数。其得名于和求最大公约数的 Euclid 算法相似的流程。

2.1 Bezout 定理

在求解 $ax + by = c$ 之前, 我们需要先判定它是否有解。

首先考虑一些较弱的结论 (必要条件)。无论 x, y 的取值如何, 左式一定是 $d = \gcd(a, b)$ 的倍数。因此, 当 $d \nmid c$ 时, 方程无解。于是, 可以将方程两侧同时除以 d , 此时 $a \perp b$ 。

根据直观感受, 如果 $a \perp b$, 那么 $ax + by$ 能组合成任何整数。只需证明 $ax \bmod b$ 可以取到 $[0, b-1]$ 的所有整数。相当于从 0 开始在长为 b 的环上每次跳 a 步。因为 $a \perp b$, 根据 1.7 小节的引理, 可以经过环上的所有点。

因此, 对任意整数 c , 存在 x 使得 $ax \equiv c \pmod{b}$ 。取 $y = \frac{c-ax}{b}$ 即可。

综上, 我们得到了 **Bezout 定理**: 二元整数线性不定方程 $ax + by = c$ 有解当且仅当 $\gcd(a, b) \mid c$ 。

2.2 扩展 Euclid 算法 (exgcd)

欲求解 $ax + by = c$, 设 $d = \gcd(a, b)$, 只需求解 $ax + by = d$, 并将解乘以 $\frac{c}{d}$ 。根据 Bezout 定理, 方程有解。

注意到等式右端等于 x, y 的系数的最大公约数。回顾 Euclid 算法, 其使用了关键结论 $\gcd(a, b) = \gcd(b, a \bmod b)$ 以减小问题规模。利用该思想, 尝试将方程也缩至更小的规模上。

因为 $d = \gcd(a, b) = \gcd(b, a \bmod b)$, 所以 $bx + (a \bmod b)y = d$ 有解。解这个方程得到 (x', y') , 则

$$\begin{aligned} ax + by = d &= bx' + (a \bmod b)y' \\ &= bx' + \left(a - b \times \left\lfloor \frac{a}{b} \right\rfloor\right) y' \\ &= ay' + b \left(x' - \left\lfloor \frac{a}{b} \right\rfloor y'\right) \end{aligned}$$

对比等式两侧, 令 $x = y'$, $y = x' - \left\lfloor \frac{a}{b} \right\rfloor y'$ 即得一组 (x, y) 。不断递归缩小问题规模直到平凡情况。当 $b = 0$ 时, 根据辗转相除法, $a = d$, 此时 $x = 1$, $y = 0$ 即为一组解。

扩展 Euclid 算法的时间复杂度是辗转相除法的 $\mathcal{O}(\log V)$ 。

```
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, x, y);
    int _y = x - (a / b) * y;
    x = y, y = _y;
}
```

也可以写成

```
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
```

- 注意: exgcd 求得的解为 $ax + by = d$ 的一组特解。为得到原方程 $ax + by = c$ 的特解, 还需将 x, y 乘以 $\frac{c}{d}$ 。

2.3 扩展

通解形式

若 $ax + by = c$ 有解, 则显然有无穷多组解。我们需要通解的形式。

使用扩欧求得特解 (x_0, y_0) 。对任意一组解 (x, y) , $ax + by = ax_0 + by_0$, 即 $(ax - ax_0) + (by - by_0) = 0$ 。根据 $\Delta(ax) + \Delta(by) = 0$, 设 $\Delta = |\Delta(ax)| = |\Delta(by)|$, 则 $a, b \mid \Delta$, 即 $\text{lcm}(a, b) \mid \Delta$ 。因此 Δx 是必须是 $\frac{\text{lcm}(a, b)}{a} = \frac{b}{d}$ 的倍数, Δy 同理。这是必要条件。

将 x 增加 $\frac{b}{d}$, y 减少 $\frac{a}{d}$, 方程仍成立, 因此条件充分, 即 Δx 为 $\frac{b}{d}$ 的倍数是通解的充要条件。故通解形如

$$\begin{cases} x = x_0 + \frac{b}{d}k \\ y = y_0 - \frac{a}{d}k \end{cases} \quad (k \in \mathbb{Z})$$

特解的数值范围

关于 `exgcd` 求得特解的数值范围, 详见 [博客](#)。结论如下: 对于非平凡情况 ($a, b \neq 0$ 且 $a \neq b$), 有 $|x| \leq \lfloor \frac{b}{2d} \rfloor$ 且 $|y| \leq \lfloor \frac{a}{2d} \rfloor$ 。

应用

- 解一元线性同余方程 $ax \equiv b \pmod{p}$ 。看成二元线性不定方程 $ax + py = b$ 使用 `exgcd` 求解。根据 Bezout 定理, 方程有解当且仅当 $\text{gcd}(a, p) \mid b$ 。
- 当模数不是质数时, Fermat 小定理不再适用, 但模意义下的乘法逆元仍可能存在。 a 在模 p 下存在逆元当且仅当 $a \perp p$, 因为求逆元相当于解 $ax \equiv 1 \pmod{p}$ 。
- $ax \equiv b \pmod{p}$ 在 $\text{gcd}(a, p) = d \nmid b$ 时无解, 否则在 $0 \sim p - 1$ 中有 d 个整数解。这是因为其通解可写为 $x_0 + k\frac{p}{d}$ 的形式, 其中 $0 \leq x_0 < \frac{p}{d}$: 将方程两侧同时除以 d , 则 $\frac{a}{d}x_0 \equiv \frac{b}{d} \pmod{\frac{p}{d}}$ 。

2.4 例题

P5656 【模板】二元一次不定方程 (exgcd)

题目要求 x 是正整数, 先求出 x 的最小正整数值 $x_0 = x \bmod \frac{b}{d}$, 此时 $y_{\max} = \frac{c - ax_0}{b}$ 。若 $y_{\max} > 0$ 则有正整数解, 且 $y_{\max}$ 对应正整数解中最大的 y , 否则没有正整数解。对于 y_0 和 $x_{\max}$ 同理。

```

#include <bits/stdc++.h>
using namespace std;
#define int long long
int T, a, b, c;
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
signed main() {
    cin >> T;
    while(T--) {
        scanf("%lld %lld %lld", &a, &b, &c);
        int d = __gcd(a, b), x, y, xx, xy, yx, yy;
        if(c % d) {
            puts("-1"); continue;
        }
        exgcd(a, b, x, y);
        x *= c / d, y *= c / d;
        xx = x % (b / d);
        if(xx <= 0) xx += b / d;
        xy = (c - a * xx) / b;
        yy = y % (a / d);
        if(yy <= 0) yy += a / d;
        yx = (c - b * yy) / a;
        if(xy <= 0) cout << xx << " " << yy << "\n";
        else cout << (yx - xx) / (b / d) + 1 << " " << xx << " " << yy << " " << yx << "
    }
    return 0;
}

```

UVA12775 Gift Dilemma

$\frac{c}{\gcd(a,b,c)} \geq 200$ 说明 $c \geq 200$ 。枚举 c 前的系数 z ，再求 $ax + by = p - cz$ 的非负整数解的个数。

记 $z = \frac{p}{c}$ ，则时间复杂度为 $\mathcal{O}(z)$ ，因为只需调用一次 `exgcd`。

CF1728E Red-Black Pepper

先通过一遍贪心对每个 i 求出选 i 个红辣椒的最大值。

对每组询问，`exgcd` 求出使得 x, y 均非负的 x 的最小整数解和最大整数解，若不存在则无解，否则三分或在极值点两侧找到最近的两个解。

时间复杂度 $\mathcal{O}((n + q) \log n)$ 。代码。

[模拟赛] 你还没有 AK 吗

给定 n, m ，求整数 $x \in [0, n]$ ， $y \in [0, m]$ ，执行任意正整数次 $x \leftarrow x + y$ 和 $y \leftarrow x + y$ 后使得 $x = X$ 的初始值 x, y 的个数。

多组数据。 $T \leq 10^5$ ， $n, m, X \leq 10^{18}$ 。

令 $f_1 = f_2 = 1$ ， $f_i = f_{i-1} + f_{i-2}$ ，则相当于求多少对 (x, y) 满足 $f_{2k+1}x + f_{2k+2}y = X$ 。

Fibonacci 数列增长很快，考虑直接枚举 k 。问题转化为求关于 x, y 的二元一次方程 $ax + by = X$ 有多少组解满足 $x \in [0, n]$ ， $y \in [0, m]$ (Fibonacci 数列相邻两项互质)。使用 `exgcd` 求解，但 $\mathcal{O}(T \log^2 X)$ 无法通过。

不难发现只有本质不同的 k 对 a, b ，直接预处理它们 `exgcd` 的结果，复杂度 $\mathcal{O}(T \log X)$ 。

进一步地，有 $f_{2k+1}^2 \equiv 1 \pmod{f_{2k+2}}$ ，所以 x 取 f_{2k+1} 即可求得一组特解，不需要使用 `exgcd`，但需要 `__int128`。

*P3518 [POI2011] SEJ-Strongbox

若 v 是密码，则所有 $\leq n$ 且是 $\gcd(v, n)$ 的倍数的数也是密码，因为 $kv \bmod n$ 取到了所有这样的数。

证明

设 $d = \gcd(v, n)$ ， $d \mid t$ 且 t 不是密码，则 $kv \equiv t \pmod{n}$ 无解。根据 Bezout 定理，它等价于 $\gcd(v, n) \nmid t$ 即 $d \nmid t$ ，矛盾。

进一步地，若 u, v 是密码，则 $u' = \gcd(u, n)$ 和 $v' = \gcd(v, n)$ 是密码。由裴蜀定理， $\gcd(u', v')$ 在模 n 下能被 $u'x + v'y$ 表出，所以 $\gcd(u', v')$ 是密码。

因此，设密码集合为 S ，则 $d = \gcd(n, \gcd_{u \in S} u) \in S$ 。显然， S 恰由 d 的所有倍数组成。

考虑枚举这个 $d = \gcd(v, n)$ ，若合法则答案即 $\max \frac{n}{d}$ ，即我们需要找到最小的合法的 d 。

设 $d_i = \gcd(m_i, n)$ ， d 必须是密码与 n 的 $\gcd$ 即 d_k 的因数，其次任何 d_i ($1 \leq i < k$) 不能是 d 的倍数。对于后者的限制，相当于在 n 的所有因数形成的图上，一个点向它的因数连边，能被某个 d_i 到达的因数是不合法的。

首先给所有 d_i ($1 \leq i \leq k$) 打上标记。从大到小枚举 n 的每个因数 c ，若 c 被打上标记，则 $\frac{c}{p_j}$ 也应被打上标记，其中 p_j 表示能整除 c 的 n 的质因数，表示若 c 是某个 d_i 的因数，则 $\frac{c}{p_j}$ 也是。

剩下没有被打标记的 n 的因数 d ，若 d 能被 d_k 整除则合法。找到最小的这样的 d ，则答案为 $\frac{n}{d}$ 。

打标记的过程可以使用哈希表实现，时间复杂度 $\mathcal{O}(\sqrt{n} + k \log n + d(n)\omega(n))$ ，其中 $\omega(i)$ 表示 i 的质因数个数。[代码](#)。

*P3421 [POI2005] SKO-Knights

题目要求我们找到两个向量 d, e ，使得它们的 **整系数** 线性组合得到的线性空间（张成）等于给出的所有向量的整系数张成。

记 $\text{span}(S)$ 表示集合 S 的整系数张成，即

$$\text{span}(S) = \left\{ \sum_{a_i \in S} c_i a_i : c_i \in \mathbb{Z} \right\}$$

Sol 1

如果我们能将三个向量 a, b, c 合并成等价的两个向量 d, e ，就能解决本题。根据基础的线性代数知识，不改变张成的初等行变换有三种：

- 交换：对应本题即任意交换 a, b, c 。
- 倍乘：对应本题即乘以任意非 0 整数。
- 倍加：对应本题即向量加上任意整数倍其它向量。

当然，上述性质仅在 **实系数** 线性组合的前提下成立。当要求系数为 **整数** 时：

- 交换显然不改变张成。
- 倍乘变换 **可能改变** 张成，因为整数乘法不可逆。如 $\text{span}\{(0, 1), (1, 0)\}$ 显然不等于 $\text{span}\{(0, 2), (1, 0)\}$ 。
- 倍加变换 **不改变** 张成。对任意 $p = y(a + xb) + zb \in \text{span}(a + xb, b)$ ，它仍是 a, b 的整系数线性组合 $ya + (xy + z)b$ 。对任意 $p = ya + zb \in \text{span}(a, b)$ ，我们也总可以将其表示为 $a + xb$ 和 b 的整系数线性组合 $y(a + xb) + (z - xy)b$ 。

很好！如果整系数倍加变换不改变整系数张成，就可以使用 **辗转相减法**。

具体地，考虑两个向量 a, b ，我们能够在不改变其整系数张成的前提下，将 b 的某一维变为 0。只需对 a, b 在这一维进行辗转相减法即可。

考虑三个向量 a, b, c ，我们对 a, b 在 x 这一维进行辗转相减，再对 a, c 在 x 这一维进行辗转相减。此时 x_b, x_c 均为 0，意味着 b, c 共线。因此，对 b, c 在 y 这一维进行辗转相减，即 $y_b \leftarrow \gcd(y_b, y_c)$ ，此时 c 变成零向量，对 $\text{span}(a, b, c)$ 没有贡献，可以直接舍去，即此时 $\text{span}(a, b, c) = \text{span}(a, b)$ 。

综上，我们在 $\mathcal{O}(n \log V)$ 的时间内解决了问题，其中 V 是值域。[代码](#)。

注意题目限制坐标绝对值不超过 10^4 。辗转相减法得到最终的 a 时, x_a 和 y_b 是原坐标的 gcd, 显然满足坐标限制。但 y_a 不一定, 因此要对 y_b 取模, 相当于做了一步辗转相减 (此时 $x_b = 0$ 所以不需要改变 x_a)。最终得到的坐标绝对值不超过 10^2 , 比题目限制优一个数量级。

从上述过程中, 我们发现一个有趣的性质: 整系数下, 两个向量可以在不改变其张成的前提下, 使其中一个向量的某一维变成 0, 而另一个向量的对应维变成原来两个向量在这一维上的 gcd。这也是解决本题的关键性质。

Sol 2

让我们深入地钻研一下题解区的其它做法。

实际上两者的核心思想是等价的: 将两个向量的其中一个的某一维变成 0, 另一个的对应维变成 gcd。不同点在于, 线性代数做法是从倍加变换和辗转相除法推得该性质, 即我们使用正确性有保证的方法, 最终得到的结果是这样的性质。而题解区的做法是首先令这一条件成立, 再根据得到的线性方程组, 使用 exgcd 和一些数学推导求解。这体现了 exgcd 与辗转相除的本质联系。

不妨设对于向量 a, b , 我们要将 x_b 变为 0。则 $x_{a'}$ 只能等于 $d_x = \gcd(x_a, x_b)$ (不管 y 坐标, 由 Bezout 定理, 能达到的 x 坐标恰为 d_x 的所有倍数)。因此, 对于方程 $x_a x + x_b y = d_x$, 使用 exgcd 求得一组特解 (x_0, y_0) , 则 $y_{a'} = y_a x_0 + y_b y_0$, 因为 $a' = x_0 a + y_0 b$ 。

对于 b' , 观察到 a' 的通解为 $(x_0 + k \frac{x_b}{d_x}, y_0 - k \frac{x_a}{d_x})$, 所以保持 $x_{a'} = d_x$ 不变, $y_{a'}$ 的最小变化量为 $\frac{y_a x_b}{d_x} - \frac{y_b x_a}{d_x}$, 即让 k 加 1 对 $y_{a'}$ 产生的变化量。因为 $x_{b'} = 0$, 所以 $y_{b'}$ 只能等于这个最小变化量。可以验证这样得到的 a', b' 符合要求。

综上, 我们将 a, b 写成了 a', b' , 其中 $x_{a'} = \gcd(x_a, x_b)$, $y_{a'}$ 用 exgcd 求解, $x_{b'} = 0$, $y_{b'} = \frac{y_a x_b - x_a y_b}{\gcd(x_a, x_b)}$ 。对 a, c 的 y 坐标做类似的操作, 再合并 b, c 即可。

Sol 3

另一种求解 $y_{b'}$ 的思路 (来源 [TheLostWeak](#)): a', b' 可以整系数线性组合出 a, b 。

考虑 a' 。因为 $x_{a'} = \gcd(x_a, x_b)$, 所以为了使横坐标等于 x_a , 需要将 a' 乘以 $\frac{x_a}{\gcd(x_a, x_b)}$ 即 $\frac{x_a}{x_{a'}}$ 的系数, 得到 $(x_a, \frac{x_a y_{a'}}{x_{a'}})$ 。因为可以通过向量 $b' = (0, y_{b'})$ 得到 (x_a, y_a) , 因此 $y_{b'} \mid y_a - \frac{x_a y_{a'}}{x_{a'}}$ 。同理 $y_{b'} \mid y_b - \frac{x_b y_{a'}}{x_{a'}}$ 。因此

$$y_{b'} = \frac{\gcd(y_a x_{a'} - x_a y_{a'}, y_b x_{a'} - x_b y_{a'})}{x_{a'}}$$

若 $y_{b'}$ 小于上述 gcd, 将导致存在 $p \in \text{span}(a', b')$, $p \notin \text{span}(a, b)$ 。若 $y_{b'}$ 大于上述 gcd, a', b' 无法整系数线性组合出 a, b 中的至少一个。

抛砖引玉：联立后两种方法描述 $y_{b'}$ 的式子，稍作化简后得到等式

$$y_a x_b - x_a y_b = \gcd(y_a x_{a'} - x_a y_{a'}, y_b x_{a'} - x_b y_{a'})$$

CF516E Drazil and His Happy Friends

不妨设 $n \perp m$ 。

如果一个男生被一个女生变得快乐，那么要么这个女生一开始就是快乐的，要么这个女生先被别的男生变得快乐。对于前者，总共只有 $O(g)$ 种情况。对于后者，对应天数可以表示为某个男生变得快乐的时间加上若干倍的 n 。因此，设 f_i 表示第 i 个男生变得快乐的时间，那么 $f_{(i+kn) \bmod m}$ 可以被 $f_i + kn$ 更新。

$i \rightarrow (i + n) \bmod m$ 连边成环，用 `exgcd` 找到初始 f_i 有值 ($f_i < n$) 的男生在环上的位置。考虑每相邻两个这样的男生之间的贡献，即相邻两个这样的男生 $i \rightsquigarrow j$ 中， j 的前驱的 f 对答案产生贡献。

男生的 f 的最大值和女生的 f 的最大值的较大值即为答案。

当 n, m 不互质时，对每个模 $d = \gcd(n, m)$ 的同余类分别求解即可。

时间复杂度 $O((b + g) \log(b + g))$ 。代码。

*P3543 [POI2012] WYR-Leveling Ground

区间操作转化为差分数组 $d_i = a_i - a_{i-1}$ ($1 \leq i \leq n + 1$) 的端点操作，其中 $a_{n+1} = 0$ 。一次 $+a$ 和一次 $-a$ 抵消，一次 $+b$ 和一次 $-b$ 抵消。设 $A = \frac{a}{d}$, $B = \frac{b}{d}$ 。

首先考虑对每个差分值单独求解，解不定方程 $ax + by = d_i$ 。设 $d = \gcd(a, b)$ ，根据 Bezout 定理，若 $d \nmid d_i$ 则无解。否则用 `exgcd` 求得 $ax + by = d$ 的特解，再乘以 $\frac{d_i}{d}$ 得到 $ax + by = d_i$ 的特解。

先不考虑可行性。因为 $ax_i + by_i = d_i$ 的贡献为 $|x_i| + |y_i|$ ，而最终答案为所有贡献之和除以 2，所以先找到 $|x_i| + |y_i|$ 最小的特解。若 x_i 和 y_i 均不为最小非负整数或最大负整数可行解，则调整法可证 $|x_i| + |y_i|$ 必然不是最优。因此我们只需检查 x_i 和 y_i 分别取到最小非负整数和最大负整数的情况。

当前 $\frac{1}{2} \sum (|x_i| + |y_i|)$ 是答案的下界，但不能保证可行性，还需满足 $X = \sum x_i = 0$ 。当 $X = 0$ 时 $Y = 0$ ，所以只需考虑 X 。

根据特解的形式

$$\begin{cases} x = x' + kB \\ y = y' - kA \end{cases}$$

当 $X < 0$ 时，我们需要进行 $-\frac{X}{B}$ 次将某个 x_i 加上 B ，并将对应的 y_i 减去 A 。 $\frac{X}{B}$ 一定是整数，因为差分数组之和为 0。类似的， $X > 0$ 时 x_i 减去 B ， y_i 加上 A 。称这样的操作为对 i 进行一次调整，调整的代价等于新的 $|x'_i| + |y'_i|$ 减去原来 $|x_i| + |y_i|$ 。

容易证明调整一个数的代价随着调整次数增加仅在 x_i 或 y_i 改变符号时增加，其它时候不变，因此我们可以用堆维护这个过程：每次取出代价最小的 i 并弹出，在需求次数与不改变符号的限制下尽可能多地调整。若经过调整后需求次数为 0，则算法结束，否则将新的调整代价压入堆。时间复杂度 $\mathcal{O}(n \log n)$ ，因为 x_i, y_i 中任意一个改变符号才会改变代价，最多发生 $2n$ 次。

一个神奇的性质是 $|\frac{X}{B}|$ 在 $\mathcal{O}(n)$ 级别，这使得我们可以从堆中取数时仅进行一次调整就塞回去。证明（来自 [评论区](#)）：当 $|x_i| + |y_i|$ 取到最小值时若 x_i 为最小非负整数或最大负整数，则 $|x_i| \leq B$ ，则 $|X| \leq nB$ 。若 y_i 为最小非负整数或最大负整数则 $|y_i| \leq A$ ，即 $\left| \frac{-XA + \sum d_i}{B} \right| \leq nA$ ，根据 $\sum d_i = 0$ 得 $|X| \leq nB$ 。

忽略问题限制得到基本解法再调整，巧妙结合反悔贪心和 exgcd。 [代码](#)。

*P7325 [WC2021] 斐波那契

解决本题需要的知识：对任意整数 m ，普通 Fibonacci 数列在模 m 下纯循环，且循环节长度为 $\mathcal{O}(m)$ 。此外， $f_{p-1} \perp f_p$ 。

$F_0 = a, F_1 = b$ 的第 p 项 $F_p = af_{p-1} + bf_p$ ，独立 a, b 前的系数可得该结果。

首先特判 $a, b = 0$ 的情况。令 $b = -b$ ，则问题等价于

$$af_{p-1} \equiv bf_p \pmod{m}$$

自然，我们希望将 b 移到方程左侧， f_{p-1} 移到方程右侧，这样可以预处理所有 $\frac{f_p}{f_{p-1}}$ 的值。但是 m 并不是质数，因此需要化简。

- 化简的核心思路是，同余方程 $A \equiv B \pmod{m}$ 等价于 $\frac{A}{d} \equiv \frac{B}{d} \pmod{\frac{m}{d}}$ ，其中 A, B 均为代数式， $d \mid A, B, m$ 。直观理解很显然，感性说明就是 d 在该同余方程中完全没有起到任何作用。

令 $d = \gcd(a, b, m)$ ，等式两侧同时除以 d ，得到

$$a'f_{p-1} \equiv b'f_p \pmod{m'}$$

其中 $a' = \frac{a}{d}, b' = \frac{b}{d}, m' = \frac{m}{d}$ 。

欲将等式两侧同时除以 b' ，即保证 b' 在模 m' 下有逆元，则需 $b' \perp m'$ 。但并不一定满足。

令 $d' = \gcd(b', m')$ 。根据既约性 ($\gcd(a', b', d') = 1$)， $d' \perp a'$ ，因此必然有 $d' \mid f_{p-1}$ ，因为方程右侧是 d' 的倍数。因此，

$$a' \frac{f_{p-1}}{d'} \equiv b'' f_p \pmod{m''}$$

其中 $b'' = \frac{b'}{d'}$ ， $m'' = \frac{m'}{d'}$ 。此时可以将 b 除过去。

$$a'(b'')^{-1}_{m''} \frac{f_{p-1}}{d'} \equiv f_p \pmod{m''}$$

欲将等式两侧同时除以 $\frac{f_{p-1}}{d'}$ ，则需 $\frac{f_{p-1}}{d'} \perp m''$ ，因为 Fibonacci 数列的性质 $f_p \perp f_{p-1}$ ，而等式右侧是 $\gcd(\frac{f_{p-1}}{d'}, m'')$ 的倍数，所以 gcd 只能等于 1。

以上推导过程说明我们需要枚举 d, d' 和 p ，并将 $(d, d', f_p(\frac{f_{p-1}}{d'})^{-1}_{m''} \bmod m'')$ 作为三元组塞入 `map`。根据一开始的结论，复杂度是 m 的所有因数的因数和，再乘以 `map` 的 `log`，无法接受。此时需要利用使得方程有解时这三个变量必须满足的性质降低复杂度。

注意到 $d' \mid f_{p-1}$ 且 $\frac{f_{p-1}}{d'} \perp \frac{m'}{d'}$ ，这说明枚举 d, p 之后， d' 只能等于 $\gcd(f_{p-1}, \frac{m}{d})$ 。时间复杂度变成 $\mathcal{O}(\sigma(m))$ 即 m 的约数和，大约是 $m \log \log m$ 量级（证明见 [ycx's blog](#)），总复杂度 $\mathcal{O}((m \log \log m + n) \log m)$ 。[代码](#)。

注意：当 f_p 或 f_{p-1} 等于 0 时，直接忽略，因为 f_p 和 f_{p-1} 不可能同时为 0 ($a, b = 0$ 的情况特判)。

总结一下，我们在化简过程中，假设了 d 和 d' 两个仅与 a, b, m 有关的变量，因此预处理时需要枚举 d 和 d' 。但根据方程有解时 d 和 d' 所具有的性质，我们得以降低复杂度。这种思想方法（在回答多组询问时，为了解决问题，需要设和询问参数相关的变量，那么在预处理时枚举这些变量的所有情况，即可考虑到给定询问参数的所有情况）是很巧妙的。

3. 线性同余方程组

前置知识：扩展 Euclid 算法。

形如

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \dots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

的方程组称为 **线性同余方程组**，其应用非常广泛（从三模 NTT 到高次剩余）。最常见的用处是将模任意合数简化为模指数幂。特别地，当模数不含平方因数时（square-free number, $\mu(p) \neq 0$ ），可以简化为模质数。

exCRT 比 CRT 更简单且适用范围更广泛，因此笔者在合并线性同余方程组时一般使用 exCRT。

3.1 中国剩余定理（CRT）

中国剩余定理（Chinese Remainder Theorem, CRT）用于求解 **模数两两互质** 的线性同余方程组。对任意 $i \neq j$ ，均有 $m_i \perp m_j$ 。

CRT 的思想是求出若干个数，使得这些数依次满足每个同余方程，且在其它模数下均为 0，则这些数的和即为所求。可以理解为 $\mathbb{Z}_M$ 上的 Lagrange 插值，其中 $M = \prod m_i$ 。

我们尝试为每个同余方程构造 $x_i \equiv a_i \pmod{m_i}$ ，记为条件一，且对于 $j \neq i$ 均有 $x_i \equiv 0 \pmod{m_j}$ ，记为条件二。

为满足条件二， x_i 必须是 $\prod_{j \neq i} m_j$ 即 $c_i = \frac{M}{m_i}$ 的倍数，令 $x_i = a_i c_i$ 。

为满足条件一，需要给 x_i 乘以 c_i 在模 m_i 下的逆元。因为 c_i 和 m_i 互质，所以 $d_i = (c_i^{-1})_{m_i}$ 存在。注意 m_i 不一定是质数，所以需要 exgcd 求逆元。于是 $x_i = a_i c_i d_i$ 。

对每个模数进行上述流程，则 $(\sum_{i=1}^k a_i c_i d_i) \bmod M$ 即为所求。时间复杂度线性对数。模板题 [P1495](#) 代码，注意使用 `__int128`。

```
#include <bits/stdc++.h>
using namespace std;
constexpr int N = 10 + 5;
int n, a[N], b[N];
long long M = 1, ans;
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
int inv(int x, int p) {
    return exgcd(x, p, x, *new int), (x % p + p) % p;
}
int main() {
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i] >> b[i], M *= a[i];
    for(int i = 1; i <= n; i++) ans = (ans + (__int128)b[i] * (M / a[i]) * inv(M / a[i] % a[i], M)) % M;
    cout << ans << endl;
    return 0;
}
```

3.2 扩展中国剩余定理 (exCRT)

扩展中国剩余定理 (exCRT) 用于求解一般形式的线性同余方程组。除了求解的问题相似，它和 CRT 并没有太大联系。

当模数不互质时，考虑合并两个同余方程 $x \equiv a_1 \pmod{m_1}$ 和 $x \equiv a_2 \pmod{m_2}$ 。因为 x 可以表示成 $a_1 + pm_1$ ，所以 $a_1 + pm_1 \equiv a_2 \pmod{m_2}$ ，即 $pm_1 + qm_2 \equiv a_2 - a_1 \pmod{m_2}$ ，exgcd 求解 p 即可。也可以理解为 $p \equiv \frac{a_2 - a_1}{m_1} \pmod{m_2}$ 并使用 exgcd 求逆元。合并后的方程为

$$x \equiv a_1 + pm_1 \pmod{\text{lcm}(m_1, m_2)}$$

根据 Bezout 定理，若 $\text{gcd}(m_1, m_2) \nmid a_2 - a_1$ ，则方程组无解。时间复杂度 $\mathcal{O}(k \log V)$ 。模板题 [P4777](#) 代码。

```
3#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll n, a, b, c, d;
void exgcd(ll a, ll b, ll &x, ll &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
int main() {
    cin >> n >> a >> b;
    for(int i = 1; i < n; i++) {
        cin >> c >> d;
        ll e = __gcd(a, c), f = (d - b % c + c) % c, inv;
        c /= e, f /= e;
        exgcd(a / e, c, inv, *new ll), inv = (inv % c + c) % c;
        b += (__int128)f * inv % c * a, a *= c, b %= a;
    }
    cout << b << endl;
    return 0;
}
```

注意考虑 exgcd 解出来是负数的情况。在得到解之后要进行取模。

*3.3 抽象代数中的 CRT

当且仅当 $n \perp m$ 时， $\mathbb{Z}/nm\mathbb{Z} \cong (\mathbb{Z}/n\mathbb{Z}) \times (\mathbb{Z}/m\mathbb{Z})$ 。这个表达式的含义是，存在二元组 (x_1, x_2) 和 x 的一一对应，其中 $x_1 \in [0, n - 1]$ ， $x_2 \in [0, m - 1]$ ， $x \in [0, nm - 1]$ ，使得对应关系保持结构。设 $f(x_1, x_2)$ 表示对应的 x ，那么

$$f(x_1, x_2) + f(y_1, y_2) = f(x_1 + y_1, x_2 + y_2), \quad f(x_1, x_2)f(y_1, y_2) = f(x_1y_1, x_2y_2)$$

实际上，一个双射由 $x \mapsto (x \bmod n, x \bmod m)$ 给出。

需要指出的是，抽象代数中的 CRT 并没有给出根据 (x_1, x_2) 求 x 的方法，且其一般形式要复杂一些。

3.4 例题

P4774 [NOI2018] 屠龙勇士

设第 i 次的攻击力为 c_i ，multiset 模拟确定。找到最小的 x 使得对所有 i 都有 $c_i x \equiv a_i \pmod{p_i}$ 且 $c_i x \geq p_i$ 。

先不管第二个条件，设 $d_i = \gcd(c_i, p_i)$ ，若 $d_i \nmid a_i$ 则无解，否则将 a_i, c_i, p_i 同时除以 d_i ，此时 $(c_i, p_i) = 1$ ，把 c_i 除过去就是标准的线性同余方程组。exCRT 求解得到 x 。

为满足第二个条件，先求出 $e_i = \lceil \frac{a_i}{c_i} \rceil$ 表示打败 i 至少需要多少次攻击。若最终求得 $x < \max e_i$ ，那么将 x 加上 $\lceil \frac{\max e_i - x}{p} \rceil \times p$ ，其中 p 是同余方程的模数。

时间复杂度 $\mathcal{O}(n \log p)$ 。代码。

P4621 [COCI2012-2013#6] BAKTERIJE

细菌的运动情况会在不超过 $4n^2 = 10^4$ 次后循环。先 BFS 找环，特判前 10^4 秒，然后 4^k 枚举每个细菌进入 (x, y) 时的方向，从而唯一确定一个线性方程组，exCRT 即可。

时间复杂度 $\mathcal{O}(4^k k \log V)$ 。代码。

P2480 [SDOI2010] 古代猪文

通过组合数学可知题目要求 $g^{\sum_{d|n} \binom{n}{d}} \bmod 999911659$ 。模数是质数，由 Fermat 小定理，指数对 999911658 取模。分解质因数后发现是 square-free number 且每个质因数较小。根据 CRT 拆成组合数对小素数取模，使用 Lucas 定理计算。

时间复杂度 $\mathcal{O}(d(n) \log n)$ 。代码。

4. 组合数取模

前置知识：组合数，二项式定理（组合数学）。

4.1 Lucas 定理

Lucas 定理是连接组合数学和数论的桥梁。

根据 1.6 小节的结论，当 p 是质数时， $\binom{n}{m} \bmod p = 0$ 当且仅当 $n - m$ 在 p 进制下需要借位，这等价于 n 在 p 进制下的每一位均不小于 m 在 p 进制下的对应位。

当 $i < j$ 时， $\binom{i}{j}$ 定义为 0。这启发我们思考：如果将 n, m 表示为 p 进制数 $n = \sum_{i=0}^c a_i p^i$ 和 $m = \sum_{i=0}^c b_i p^i$ （不足的位补 0），那么是否有 $\binom{n}{m} \equiv \prod_{i=0}^c \binom{a_i}{b_i} \pmod{p}$ ？

引理

当 p 是质数时

$$(x + y)^p \equiv x^p + y^p \pmod{p}$$

证明

使用二项式定理将上式拆开，当 $1 \leq i < p$ 时， $\binom{p}{i}$ 是 p 的倍数。□

推论

$$(1 + x)^p \equiv 1 + x^p \pmod{p}$$

Lucas 定理

当 p 是质数时，

$$\binom{n}{m} \equiv \binom{\lfloor \frac{n}{p} \rfloor}{\lfloor \frac{m}{p} \rfloor} \binom{n \bmod p}{m \bmod p} \pmod{p}$$

证明

设 $n = dp + r$ ($0 \leq r < p$)，则

$$(1 + x)^n = (1 + x)^{dp} (1 + x)^r \equiv (1 + x^p)^d (1 + x)^r \pmod{p}$$

由二项式定理，上式 x^m 前的系数即 $\binom{n}{m}$ 。

因为 $(1 + x^p)^d$ 展开后 x 的次数均为 p 的倍数， $(1 + x)^r$ 展开后 x 的次数小于 p ，因此 x^m 项只能由 $(1 + x^p)^d$ 展开后的 $x^{\lfloor m/p \rfloor p}$ 项与 $(1 + x)^r$ 展开后的 $x^{m \bmod p}$ 项相乘得到。□

另一种形式

设 n, m 在 p 进制下为 $\sum_{i=0}^c a_i p^i$ 和 $\sum_{i=0}^c b_i p^i$ ，则

$$\binom{n}{m} \equiv \prod_{i=0}^c \binom{a_i}{b_i} \pmod{p}$$

显然，两种形式是等价的。

根据 Lucas 定理，计算大组合数对 p 取模可拆成计算小组合数。 $\mathcal{O}(p)$ 预处理阶乘及其逆元做到 $\mathcal{O}(1)$ 计算组合数，时间复杂度 $\mathcal{O}(\log_p n)$ 。模板题 [P3807](#) 代码。

```
#include <bits/stdc++.h>
using namespace std;
constexpr int N = 1e5 + 5;
int T, n, m, p, fc[N], ifc[N];
int ksm(int a, int b) {
    int s = 1;
    while(b) {
        if(b & 1) s = 1ll * s * a % p;
        a = 1ll * a * a % p, b >>= 1;
    }
    return s;
}
int binom(int n, int m) {
    return 1ll * fc[n] * ifc[m] % p * ifc[n - m] % p;
}
int lucas(int n, int m) {
    if(n % p < m % p) return 0;
    if(n < p) return binom(n, m);
    return 1ll * lucas(n / p, m / p) * binom(n % p, m % p) % p;
}
void solve() {
    cin >> n >> m >> p;
    for(int i = fc[0] = 1; i < p; i++) fc[i] = 1ll * fc[i - 1] * i % p;
    ifc[p - 1] = ksm(fc[p - 1], p - 2);
    for(int i = p - 2; ~i; i--) ifc[i] = 1ll * ifc[i + 1] * (i + 1) % p;
    cout << lucas(n + m, n) << endl;
}
int main() {
    cin >> T;
    while(T--) solve();
    return 0;
}
```

在 p 进制下考虑组合数取模是很有帮助的。计算大组合数对 p 取模的值，可以将 n, m 在 p 进制下的每一位分开考虑，再将所得结果相乘。特别地，若 n 在 p 进制下至少有一位小于 m ，则 $\binom{n}{m} \equiv 0 \pmod{p}$ ，因为对 $a_i < b_i$ 有 $\binom{a_i}{b_i} = 0$ 。这符合 1.6 小节的结论。

Lucas 定理还可以快速计算组合数奇偶性。 $\binom{n}{m}$ 是奇数当且仅当 m 在二进制下每一位均不大于 n ，当且仅当 n 和 m 的按位与等于 m ，当且仅当 n 和 m 的按位或等于 n 。

4.2 扩展 Lucas 定理 (exLucas)

当模数 p 不是质数时，也有快速计算 $\binom{n}{m} \bmod p$ 的方法。

模合数的问题的第一步一定是 CRT，将问题简化为模指数幂的情况。具体地，设 $p = \prod q_i^{k_i}$ ，分别计算答案模 $q_i^{k_i}$ 的结果，再使用 CRT 合并。这就是 CRT 的神力！

考虑 $\binom{n}{m} \bmod q^k$ ，自然地把组合数写成 $\frac{n!}{m!(n-m)!}$ 。因为 $m!(n-m)!$ 含有质因数 q ，所以无法求逆元。但因为模数只含质因数 q ，所以我们考虑把 q 和其它质因数分开来算：求出组合数含有多少 q ，以及分子和分母除掉其所包含的所有 q 之后的结果，即变形为

$$\frac{\frac{n!}{q^{v_q(n!)}}}{\frac{m!}{q^{v_q(m!)}} \frac{(n-m)!}{q^{v_q((n-m)!)}}} \times q^{v_q(n!) - v_q(m!) - v_q((n-m)!)}$$

使用 Kummer 定理计算组合数的质因数幂次 $(v_q(n!) - v_q(m!) - v_q((n-m)!))$ 。此时分子和分母均不含 q ，分别计算，再对分母求逆元即可。问题转化为 $\frac{n!}{q^{v_q(n!)}} \bmod q^k$ ，也就是阶乘去掉所有 q 之后对 q^k 取模。

设 $d = \lfloor \frac{n}{q} \rfloor$ ， $d_k = \lfloor \frac{n}{q^k} \rfloor$ ， $r = n \bmod q^k$ 。既然除掉了所有 q ，自然想到将所有数分成 q 的倍数和不是 q 的倍数讨论。

对于 q 的倍数，有 $q, 2q, \dots, dq$ ，我们给每个数除掉 q ，问题变成求 $\frac{d!}{q^{v_q(d!)}}$ 。这是子问题。

对于不是 q 的倍数，把它们全部模掉 q^k ，得到 d_k 组 $1 \sim q^k$ 内和 q 互质的数，以及一组 $1 \sim r$ 内和 q 互质的数。回忆扩展 Wilson 定理的符号， $(n!)_q$ 表示 $1 \sim n$ 所有和 q 互质的数的乘积，所以只需计算 $((q^k!)_q^{d_k} \times (r!)_q) \bmod q^k$ 。预处理 q^k 以内所有与 q 互质的数的前缀积，这部分可以 $\mathcal{O}(\log n)$ 计算（快速幂）。进一步地，根据扩展 Wilson 定理， $(q^k!)_q \bmod q^k$ 等于 1 或 -1 ，所以不需要快速幂。

综上，

$$\frac{n!}{q^{v_q(n!)}} \equiv \frac{d!}{q^{v_q(d!)}} \times (q^k!)_q^{d_k} \times (r!)_q \pmod{q^k}$$

当 $n < q$ 时，直接返回预处理结果，否则使用递归式计算。时间复杂度 $\mathcal{O}(\log_q n)$ 。

- 注意：递归的子问题是除以 q ，而循环节数量是除以 q^k 。
- 注意：区分 $\frac{n!}{q^{v_q(n!)}}$ 和 $(n!)_q$ 。前者只是将所有 p 去掉，而后者将含有质因数 p 的所有数去掉了。

综上，我们在 $\mathcal{O}(\omega(p) \log n + \sum q_i^{k_i})$ 的时间内求出一个大组合数模任意数 p 的值。若模数固定且预处理，则可以做到单次 $\mathcal{O}(\omega(p) \log n)$ 。模板题 [P4720](#) 代码。

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
int inv(int x, int p) {
    return exgcd(x, p, x, *new int), (x % p + p) % p;
}
ll n, m;
int p, tp, ans, curp = 1, q, qk, fc[1000005];
ll num(ll n) {
    ll s = 0;
    while(n) s += n /= q;
    return s;
}
int calc(ll n) {
    if(!n) return 1;
    return 1ll * calc(n / q) * (tp && (n / qk & 1) ? qk - 1 : 1) % qk * fc[n % qk] % qk;
} // 注意 n / q 和 n / qk!
int solve() {
    tp = (qk & 1) || qk <= 4; // 扩展 Wilson 定理.
    for(int i = fc[0] = 1; i < qk; i++) fc[i] = 1ll * fc[i - 1] * (i % q ? i : 1) % qk;
    int pw = num(n) - num(m) - num(n - m);
    int res = 1ll * calc(n) * inv(1ll * calc(m) * calc(n - m) % qk, qk) % qk;
    while(pw--) res = 1ll * res * q % qk; // 根据 Kummer 定理, 质因子在组合数中的个数为对数级别.
    return res;
}
int main() {
    cin >> n >> m >> p;
    for(int i = 2; i <= p; i++) {
        if(i * i > p) i = p;
        if(p % i == 0) {
            q = i, qk = 1;
            while(p % i == 0) qk *= i, p /= i;
            ans += 1ll * (solve() - ans % qk + qk) * inv(curp, qk) % qk * curp;
            ans %= (curp *= qk); // exCRT 合并更方便.
        }
    }
    cout << ans << endl;
    return 0;
}

```

4.3 例题

[BZOJ2445] 最大团

定义一张 n 个点的图是“好图”当且仅当它能被分成若干大小相等的完全图。结点有标号。记 n 个点的好图个数为 $G(n)$ 。求 $m^{G(n)} \bmod 999999599$ 。 $n, m \leq 2 \times 10^9$ 。

模数是质数，Fermat 小定理转化为求 $G(n) \bmod (10^9 - 402)$ 。

被分成的完全图大小 d 是 n 的因数，考虑枚举 $d \mid n$ 并计算完全图大小为 d 时的方案数。

根据组合意义，从 n 个节点中选出 d 个点连成完全图，再从剩下 $n - d$ 个节点中选出 d 个点连成完全图，以此类推。最后剩下的 d 个节点连成完全图。由于完全图之间无序，而我们枚举完全图的过程有序，因此要除以 $\frac{n}{d}$ 的阶乘。记 $c = \frac{n}{d}$ ，则

$$ans = \frac{1}{c!} \binom{n}{d, d, \dots, d} = \frac{n!}{(d!)^c \times c!}$$

观察 99999598，将其分解质因数后得到 $2 \times 13 \times 5281 \times 7283$ ，模数是 square-free number。考虑分别求上式对这四个质数取模后的结果，再 CRT 合并。问题转化为计算答案对质数 p 取模后的结果，其中 p 较小。

根据 1.6 小节的结论，先求出答案含有多少个质因子 p ，若个数不为 0 则直接返回 0。否则问题转化为计算一个数的阶乘去掉所有质因子 p 后模 p 的结果，使用 exLucas 即可。[代码](#)。

*P5598 【XR-4】 混乱度

设 $S(l, r)$ 表示 a 的区间和，根据组合意义得到

$$f(l, r) = f(l, r - 1) \binom{S(l, r)}{a_r} = f(l, r) \binom{S(l, r)}{S(l, r - 1)}$$

固定 l ，考虑 r 在增大的过程中， $S(l, r)$ 不断增加。根据 Lucas 定理，一旦 $S(l, r)$ 在 p 进制下进位，那么之后的 $f(l, r) \bmod p$ 均为 0。因此，在经过至多 $p \log_p V$ 个非零的 a_r 后， $f(l, r)$ 对答案不再有贡献。

预处理每个非零的 a_r 在 p 进制下有哪些位非零以及这一位上的值，对每个 l ，维护当前 $S(l, r)$ 在 p 进制下每一位的值，可快速计算 $\binom{S(l, r)}{S(l, r - 1)}$ 。考虑到 $a_r = 0$ 时 $f(l, r - 1) = f(l, r)$ ，所以对每个位置预处理下一个非零的位置即可快速跳过等于 0 的位置。

时间复杂度 $\mathcal{O}(np \log_p n)$ 。[代码](#)。

5. 离散对数问题

离散对数问题即求解 **离散对数方程**

$$a^x \equiv b \pmod{p}$$

其中 x 即模 p 下的 $\log_a b$ 。解可能不存在，也可能存在多个，我们只要求出任意一个。

5.1 大步小步算法 (BSGS)

大步小步算法 (Baby Step Giant Step, BSGS) 要求 $a \perp p$ 。

BSGS 使用了 meet-in-the-middle: 设块长为 M 且 $x = AM - B$, 则 $a^{AM} \equiv ba^B \pmod{p}$ 。因为 $a \perp p$, 所以若 $a^{AM} \equiv ba^B \pmod{p}$, 则 $a^{AM-B} \equiv b \pmod{p}$ 。枚举 a^{AM} 与 ba^B 所有可能的取值, 并通过哈希表判断是否存在 A, B 使得这两个值相等。时间复杂度 $\mathcal{O}(\max(A, B))$ 。

根据 Euler 定理, 若有解, 则在 $[0, p-2]$ 上有解, 所以 $1 \leq B \leq M, 1 \leq A \leq \lceil \frac{p-1}{M} \rceil$ 。将阈值 M 设为 $\sqrt{p}$ 得到最优复杂度 $\mathcal{O}(\sqrt{p})$ 。模板题 [P3846](#) 代码。

```
#include <bits/stdc++.h>
using namespace std;
int BSGS(int a, int b, int p) {
    int A = 1, B = sqrt(p) + 1;
    a %= p, b %= p;
    unordered_map<int, int> mp;
    for(int i = 1; i <= B; i++) {
        A = 1ll * A * a % p;
        mp[1ll * A * b % p] = i;
    }
    for(int i = 1, AA = A; i <= B; i++) {
        if(mp.find(AA) != mp.end()) {
            return i * B - mp[AA];
        }
        AA = 1ll * AA * A % p;
    }
    return -1;
}
int main() {
    int p, b, n;
    cin >> p >> b >> n;
    int ans = BSGS(b, n, p);
    if(~ans) cout << ans << "\n";
    else puts("no solution");
    return 0;
}
```

根据枚举顺序可知以上代码求得 $\log_a b$ 的最小的非负整数解。解的循环节长度为最小的正整数 y 使得 $a^y \equiv 1 \pmod{p}$ ，即 $\delta_p(a)$ ，见 6.1 小节。

5.2 扩展大步小步算法 (exBSGS)

对于更一般的离散对数方程，没有 $a \perp p$ 的条件，该怎么办呢？

从已知到未知。既然可以解决 $a \perp p$ ，那么我们尝试把 a, p 凑成互质。同余式两侧同时除以 $d = \gcd(a, p)$ ，方程化为 $\frac{a}{d} a^{x-1} \equiv \frac{b}{d} \pmod{\frac{p}{d}}$ 。若 $d \nmid b$ 显然无解。

注意到 $\frac{a}{d} \perp \frac{p}{d}$ ，所以前者在模后者下存在逆元。 p 可能不是质数，需要使用 exgcd 求逆元。移项，方程变为 $a^{x-1} \equiv \frac{b}{d} \times \left(\frac{a}{d}\right)^{-1} \pmod{\frac{p}{d}}$ ，等式右边为常数。

因为 a 和 $\frac{p}{\gcd(a, p)}$ 可能不互质，所以重复上述操作直到 a, p 互质，此时直接使用 BSGS 求解即可。设提出的 a 的数量为 k ，则求出的解需要加上 k 。注意在每次操作开始前特判 $b = 1$ ，此时答案恰等于已经提出的 a 的数量。

显然 $k = \mathcal{O}(\log p)$ ，时间复杂度 $\mathcal{O}(\sqrt{p})$ 。模板题 [SP3105](#) 代码。

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
using namespace std;
using namespace __gnu_pbds;
int a, p, b;
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
int inv(int x, int p) {return exgcd(x, p, x, *new int), (x % p + p) % p;}
int BSGS(int a, int b, int p) {
    a %= p, b %= p;
    int A = 1, B = sqrt(p) + 1;
    gp_hash_table<int, int> mp;
    for(int i = 1; i <= B; i++) mp[1ll * (A = 1ll * A * a % p) * b % p] = i;
    for(int i = 1, AA = A; i <= B; i++, AA = 1ll * AA * A % p) if(mp.find(AA) != mp.end()) r
    return -1;
}
int exBSGS(int a, int b, int p) {
    int d = __gcd(a, p), cnt = 0;
    a %= p, b %= p;
    while(d > 1) {
        if(b == 1) return cnt;
        if(b % d) return -1;
        p /= d, b = 1ll * b / d * inv(a / d, p) % p;
        d = __gcd(p, a %= p), cnt++;
    }
    int ans = BSGS(a, b, p);
    return ~ans ? ans + cnt : -1;
}
int main() {
    cin >> a >> p >> b;
    while(a) {
        int ans = exBSGS(a, b, p);
        if(~ans) cout << ans << "\n";
        else puts("No Solution");
        cin >> a >> p >> b;
    }
    return 0;
}

```

5.3 例题

P2485 [SDOI2011] 计算器

操作 1 快速幂，操作 2 exgcd 求逆元，操作 3 BSGS。

P3306 [SDOI2013] 随机数生成器

把 x 的坐标左移一位，显然有 $x_i = a^i x_0 + b \sum_{j=0}^{i-1} a^j$ 。由等比数列求和公式，

$$x_i = a^i x_0 + \frac{a^i - 1}{a - 1} \times b = a^i \left(x_0 + \frac{b}{a - 1} \right) - \frac{b}{a - 1}$$

对同余式进行简单变形后得到离散对数问题，使用 BSGS 求解。时间复杂度 $\mathcal{O}(T\sqrt{p})$ 。代码。

6. 阶与原根

对于含乘法的同余式 $ab \equiv c \pmod{m}$ ，如果能找到 g, x, y, z 使得 $g^x \equiv a \pmod{m}$, $g^y \equiv b \pmod{m}$, $g^z \equiv c \pmod{m}$ ，那么根据 Euler 定理，同余式等价于 $x + y \equiv z \pmod{\varphi(m)}$ 。也就是说，我们希望找到一个共同的底，将幂次的乘法化为指数上的加法。

在模数是合数时，一般会将问题化为只考虑和模数互质的数。 m ($m \geq 2$) 的简化剩余系在模 m 的乘法下构成群（封闭、结合律、逆元、单位元）。借用群相关的定义，得到阶和原根（循环群的生成元）的概念。

对 $a \perp m$ ，如果 a 的幂次模 m 可以取到整个简化剩余系，那么称 a 是 m 的原根。可以发现 a 就是上述问题的一个较为可行的底，因为所有和 m 互质的数都可以用它的幂次表示。

6.1 阶及其性质

定义

$a^x \equiv 1 \pmod{m}$ 的最小正整数 x 称为 a 模 m 的 **阶**，记作 $\delta_m(a)$ 。 $a \perp m$ 是 $\delta_m(a)$ 存在的充要条件。

必要性

若 a 与 m 不互质，则 a^x 与 m 不互质，模 m 不可能得 1。

充分性

若 $a \perp m$ ，根据 Euler 定理， $x = \varphi(m)$ 即一解，因此阶存在。

以下设 $a \perp m$ 。

性质

阶本质上在刻画 a 的幂次的循环节， a 的幂次模 m 是纯循环。如果 $a^x \equiv a^y \pmod{m}$ ，则 $a^{y-x} \equiv 1 \pmod{m}$ 。由此我们知道 $a^0, a^1, \dots, a^{\delta_m(a)-1}$ 在模 m 下一定是互不相同的，否则会跟 $\delta_m(a)$ 的最小性矛盾。于是 a 的幂次模 m 一定是两两不同的 $a^0, a^1, \dots, a^{\delta_m(a)-1}$ 的不断循环。

更具体地， $a^x \equiv a^y \pmod{m}$ 当且仅当 $x \equiv y \pmod{\delta_m(a)}$ 。

性质 1

$a^x \equiv 1 \pmod{m}$ 当且仅当 $\delta_m(a) \mid x$ 。

证明

若 $a^x \equiv 1 \pmod{m}$ ，则 $a^{kx} \equiv 1 \pmod{m}$ 。因此若 $\delta_m(a) \mid x$ ，则 $a^x \equiv 1 \pmod{m}$ 。

若 $\delta_m(a) \nmid x$ ，设 $r = x \bmod \delta_m(a)$ ，则 $a^x \equiv a^r \pmod{m}$ 。因为 $0 < r < \delta_m(a)$ ，所以 $a^r \not\equiv 1 \pmod{m}$ 。□

考虑一个数自身相乘时，它的阶如何变化。如果我们将模 m 下 a 的幂次看成一个长为 $\delta_m(a)$ 的环，那么 a 的幂次相当于在环上每次跳一步， a^k 的幂次相当于在环上每次跳 k 步。使用 1.7 小节的结论即可。

性质 2

$$\delta_m(a^k) = \frac{\delta_m(a)}{\gcd(\delta_m(a), k)} = \frac{\text{lcm}(\delta_m(a), k)}{k}$$

证明

注意到 $(a^k)^x = a^{kx}$ 。

对于第一部分，根据性质 1， $\delta_m(a^k)$ 为最小的 x 使得 $\delta_m(a) \mid kx$ 。于是 $x_{\min} = \frac{\delta_m(a)}{\gcd(\delta_m(a), k)}$ （见最开头的基本知识）。

对于第二部分，同时是 k 和 $\delta_m(a)$ 的倍数的最小正整数为 $\text{lcm}(\delta_m(a), k)$ ，所以使得 $a^{kx} \equiv 1 \pmod{m}$ 的最小的 kx 即 $\text{lcm}(\delta_m(a), k)$ 。于是 $x_{\min} = \frac{\text{lcm}(\delta_m(a), k)}{k}$ 。

以上两部分也可以根据 $ab = \gcd(a, b) \times \text{lcm}(a, b)$ 相互推导。□

考虑两个数相乘时，它们的阶如何变化。和一个数自身的幂次 $\delta_m(a^k) \leq \delta_m(a)$ 不一样，乘积的阶可能小于其中每个因子的阶，也可能大于。我们需要知道这种变化是由什么引起的。从循环节的角度考虑会带给我们一些启发：乘积的阶变小是由于其两个因子的阶的公共因数，乘积的阶变大是由于其两个因子的阶的非公共因数。

如果 $\delta_m(a) = \delta_m(b)$ ，那么 $\delta_m(ab)$ 可以是 $\delta_m(a)$ 的任何因数。 $\delta_m(g) = \delta_m(g^2) = \delta_m(g^4) = \delta_m(g^{14}) = 15$ ，而 $\delta_m(g \cdot g^{14}) = 1$ ， $\delta_m(g \cdot g^4) = 3$ ， $\delta_m(g \cdot g^2) = 5$ ， $\delta_m(g \cdot g) = 15$ 。

性质 3

$\delta_m(ab) = \delta_m(a)\delta_m(b)$ 当且仅当 $\delta_m(a) \perp \delta_m(b)$ 。

证明

必要性是显然的。

充分性的证明思路是注意到对所有 $\delta_m(a)$ 的倍数或 $\delta_m(b)$ 的倍数但不为 $\text{lcm}(\delta_m(a), \delta_m(b)) = \delta_m(a)\delta_m(b)$ 的倍数 x ，对于 $a^x \bmod m$ 和 $b^x \bmod m$ ，一定恰有一个等于 1，另一个不等于 1，所以 x 一定不是 $\delta_m(ab)$ 的倍数。

两个最小循环节分别为 $\delta_m(a)$ 和 $\delta_m(b)$ 的序列相乘，一定有 $L = \delta_m(a)\delta_m(b)$ 的循环节，最小循环节显然是 L 的因数。如果最小循环节不等于 L ，那么存在 L 的质因数 p 使得 $(ab)^{L/p} \equiv 1 \pmod{m}$ 。不妨设 $p \mid \delta_m(b)$ ，则 $\frac{L}{p} = k\delta_m(a)$ ，其中 $k = \frac{\delta_m(b)}{p}$ 。因为 $\delta_m(a) \perp \delta_m(b)$ ，所以 $k\delta_m(a)$ 不是 $\delta_m(b)$ 的倍数。但 $1 \equiv (ab)^{k\delta_m(a)} \equiv b^{k\delta_m(a)} \pmod{m}$ ，矛盾。这说明最小循环节等于 L 。□

对于更一般的情况呢？读者可以先结合以上两种特殊情况自行思考。

性质 4

对于质因数 p ，如果它在 $\delta_m(a)$ 和 $\delta_m(b)$ 的次数相等，那么它在 $\delta_m(ab)$ 的次数是任意的，但不会超过它在 $\delta_m(a)$ 的次数。如果不等，那么它在 $\delta_m(ab)$ 的次数是它在 $\delta_m(a)$ 和 $\delta_m(b)$ 的次数的较大值。

可以这样类比：设 p 在 a 的次数为 x ，在 b 的次数为 y 。如果 $x = y$ ，那么 p 在 $a + b$ 的次数 $z \geq x$ 可以任意大。如果 $x \neq y$ ，那么 $z = \min(x, y)$ 。至于为什么是加法，考虑到阶本身是描述指数循环节的，而相乘转化到指数上就是加法。

如果 m 有原根 g ，那么一切都是好理解的： $\delta_m(ab) = \frac{\varphi(m)}{\gcd(\varphi(m), \log_g a + \log_g b)}$ ，指数在分母上解释了一个是任意小一个是任意大，一个是较大值一个是较小值。

性质 4 描述了乘积的阶与每个因子的阶的关系。

性质 2 和性质 4 给予我们操作阶的空间，并引出了以下性质。

性质 5

给定 $a, b \perp m$ ，存在 c 使得 $\delta_m(c) = \text{lcm}(\delta_m(a), \delta_m(b))$ 。

证明

对每个质因数 p ，如果 p 在 $\delta_m(a), \delta_m(b)$ 的次数相等且不为 0，那么让 $a \leftarrow a^p$ 。由性质 2，因为 $p \mid \delta_m(a)$ ，所以 $\delta_m(a^p) = \frac{\delta_m(a)}{p}$ 。处理后 $\delta_m(a')$ 和 $\delta_m(b)$ 没有次数相同的质因数。根据性质 4 可知 $\delta_m(a'b) = \text{lcm}(\delta_m(a), \delta_m(b))$ 。□

离散对数的通解形式：当 $a \perp m$ 时，使得 $a^x \equiv b \pmod{m}$ 的 x 若存在，则其循环节恰为 $\delta_m(a)$ 。可以求出最小和次小的解 x_1, x_2 ，则通解形式为 $x = x_1 + k(x_2 - x_1) \ (k \in \mathbb{N})$ 。也可以先把阶求出来。

求法

法一：使用 BSGS 求 $a^x \equiv 1 \pmod{m}$ 的最小正整数解。

法二：根据性质 1，对 $x = k\delta_m(a)$ 有 $a^x \equiv 1 \pmod{m}$ 。考虑令 $x = \varphi(m)$ ，对于 $\varphi(m)$ 的每个质因子 p ，用 x 不断试除 p 直到 x 无法整除 p 或 $a^{x/p} \not\equiv 1 \pmod{m}$ 。时间复杂度为分解质因数加上 $\mathcal{O}(\log^2 m)$ 。

6.2 原根

以下设 $m \geq 2$ 且 $a \perp m$ 。

定义

若 $\delta_m(a) = \varphi(m)$ ，则称 a 为模 m 的 **原根**。并不是所有数都有原根。存在原根的模 m 的缩系同构于循环群：原根等价于循环群的生成元。

我们需要原根是因为它可以 **生成** 模 m 的缩系，即它的若干次幂在模 m 下取遍了所有与 m 互质的数。这使得我们在考虑解与模数互质且模数存在原根的同余方程时可以用原根的若干次幂代替未知数，在求解高次剩余时非常有用。

简单地说，它可以将乘法转化为加法。

性质

判定原根的一般方法是直接计算阶。如果 $\delta_m(a) \neq \varphi(m)$ ，那么一定存在 $\varphi(m)$ 的质因数 p 使得 $\frac{\varphi(m)}{p}$ 是 $\delta_m(a)$ 的倍数，即 $a^{\varphi(m)/p} \equiv 1 \pmod{m}$ 。

原根判定定理

a 是 m 的原根当且仅当对任意 $\varphi(m)$ 的质因子 p ，均有 $a^{\varphi(m)/p} \not\equiv 1 \pmod{m}$ 。

除了判断一个数是否是原根，我们还需要判断一个数是否有原根。

原根存在定理

m 有原根当且仅当 $m = 2, 4, p^\alpha, 2p^\alpha$, 其中 p 是奇质数。

证明较复杂, 不感兴趣的读者可以跳过。

2, 4 有原根是显然的。

奇质数

对奇质数 p , 由引理 5, 存在 g 使得对任意 $1 \leq i < p$, $\delta_p(i) \mid \delta_p(g)$, 所以 $x^{\delta_p(g)} \equiv 1 \pmod{p}$ 有 $p-1$ 个不同的解。由 Lagrange 定理, $\delta_p(g) = p-1$ 。

奇质数幂

考虑数学归纳。

设 g 是奇质数 p 的原根。如果 $g^{p-1} \equiv 1 \pmod{p^2}$, 那么

$$(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p \equiv 1 - pg^{-1} \not\equiv 1 \pmod{p^2}$$

设 g 是奇质数幂 p^α 的原根, $1 \leq k < p$, 且 $g^{p^{\alpha-1}(p-1)} \equiv 1 + kp^\alpha \pmod{p^{\alpha+1}}$ (当 $\alpha = 1$ 时, 上述说明保证这样的 g 存在), 那么

$$g^{p^{\alpha-1}(p-1)} \equiv 1 + (k+cp)p^\alpha \pmod{p^{\alpha+2}}$$

于是

$$g^{p^\alpha(p-1)} \equiv (1 + (k+cp)p^\alpha)^p \equiv 1 + kp^{\alpha+1} \pmod{p^{\alpha+2}}$$

且 g 是奇质数幂 $p^{\alpha+1}$ 的原根。由数学归纳法可知 g 是任意 p^α ($\alpha \geq 1$) 的原根。

- 请思考以上推导过程在哪里对 $p = 2$ 不适用。

设 g 是 p^α 的原根, 考虑 $g \bmod p^\alpha$ 可知 $\delta_{2p^\alpha}(g) \geq \delta_{p^\alpha}(g) = \varphi(p^\alpha) = \varphi(2p^\alpha)$ 。所以 g 是 $2p^\alpha$ 的原根。

必要性

首先, 如果 m 有原根, 那么 m 的所有因数都有原根。所以只需证明 8, $4p$ 和 p_1p_2 没有原根。8 没有原根是显然的。

对任意 $x \perp 4p$, 因为 $\varphi(p)$ 是 $\varphi(4) = 2$ 的倍数, 所以 $x^{\varphi(p)} \equiv 1 \pmod{p}$ 且 $x^{\varphi(p)} \equiv 1 \pmod{4}$ 。由 CRT, $x^{\varphi(p)} \equiv 1 \pmod{4p}$ 。

对任意 $x \perp p_1 p_2$, 因为 $\varphi(p_1), \varphi(p_2)$ 都是 2 的倍数, 所以 $x^{\varphi(p_1 p_2)/2} \equiv 1 \pmod{p_1}$ 且 $x^{\varphi(p_1 p_2)/2} \equiv 1 \pmod{p_2}$ 。由 CRT, $x^{\varphi(p_1 p_2)/2} \equiv 1 \pmod{p_1 p_2}$ 。□

如果一个数有原根, 那么其缩系上的乘法等价于模 $\varphi(m)$ 下的加法: 将缩系的每个数变成原根的幂次, 两个数相乘是同底数幂相乘, 指数相加。这提供了更多阶的性质。

性质 6

若 m 有原根, 则使得 $\delta_m(a) = l$ 的 a 的个数为

$$\begin{cases} 0 & l \nmid \varphi(m) \\ \varphi(l) & l \mid \varphi(m) \end{cases}$$

证明

根据性质 1, 第一部分显然。考虑第二部分。

设 g 是 m 的原根。因为任意 $a \perp m$ 均可表示为 g^k ($0 \leq k < \varphi(m)$), 所以答案即使得 $\delta_m(g^k) = l$ 的 k 的个数。根据性质 2, $\delta_m(g^k) = \frac{\delta_m(g)}{\gcd(\delta_m(g), k)}$, 因此 $\frac{\varphi(m)}{\gcd(\varphi(m), k)} = l$, 即 $\gcd(\varphi(m), k) = \frac{\varphi(m)}{l}$ 。由基本知识, k 的个数为 $\varphi(\frac{\varphi(m)}{\varphi(m)/l}) = \varphi(l)$ 。□

形象地, 将 $g^k \bmod m \rightarrow g^{k+1} \bmod m$ 看成长度为 $\varphi(m)$ 的环。在环上每次走 k 步形成子环, 则 g^k 的阶等于子环环长。由 1.7 小节, 环长为 $\frac{n}{\gcd(n, k)}$ 。为使 $\frac{\varphi(m)}{\gcd(\varphi(m), k)} = l$, 则 $\gcd(\varphi(m), k) = \frac{\varphi(m)}{l}$ 。

性质 6 反映了欧拉函数的性质 $\sum_{d|n} \varphi(d) = n$ 。 $\sum_{l|\varphi(m)} \varphi(l) = \varphi(m)$, 即模 m 的阶等于 l 的数的个数之和 (模 m 下存在阶的数的个数) 等于和 m 互质的数的个数, 而和 m 互质是在模 m 下存在阶的充要条件。

原根个数定理

若正整数 m 有原根, 则原根个数为 $\varphi(\varphi(m))$ 。

求法

求一个数的所有原根: 求出任意一个原根 g , 根据性质 2, $g^k \bmod m$ ($k \perp \varphi(m)$) 也是 m 的原根。相当于在长为 $\varphi(m)$ 的环上每次走 k 步, 因为 $k \perp \varphi(m)$, 所以一定走遍整个环。

中国数学家王元证明了质数的最小原根的数量级为 $\mathcal{O}(m^{0.25+\epsilon})$ (1959, 需要指出的是大 $\mathcal{O}$ 符号仅表示上界)。结合原根存在定理的推导过程, 可以证明任何有原根的数最小原根不超过这个数量级。再根据原根判定定理, 可以 $\mathcal{O}(m)$ 预处理每个数的最小质因子后 $\mathcal{O}(m^{0.25}\omega(m)\log m)$ 求出 m 的最小原根, 从而求出 m 的所有原根。模板题 [P6091](#) 代码。

```

#include <bits/stdc++.h>
using namespace std;
constexpr int N = 1e6 + 5;
int ksm(int a, int b, int p) {
    int s = 1;
    while(b) {
        if(b & 1) s = 1ll * s * a % p;
        a = 1ll * a * a % p, b >>= 1;
    }
    return s;
}
void solve() {
    int n, d;
    cin >> n >> d;
    int x = n, phi = n;
    for(int i = 2; i <= x; i++) {
        if(x % i == 0) {
            phi = phi / i * (i - 1);
            while(x % i == 0) x /= i;
        }
    }
    vector<int> pr;
    x = phi;
    for(int i = 2; i <= x; i++) {
        if(x % i == 0) {
            pr.push_back(i);
            while(x % i == 0) x /= i;
        }
    }
    for(int i = 1; i < n; i++) {
        if(__gcd(i, n) != 1) continue;
        bool ok = 1;
        for(int it : pr) {
            ok &= ksm(i, phi / it, n) != 1;
        }
        if(ok) {
            static int is[N], g[N];
            memset(is, 0, N << 2);
            is[i] = 1;
            for(int j = 2; j <= phi; j++) {
                if(__gcd(j, phi) == 1) {
                    is[ksm(i, j, n)] = 1;
                }
            }
            int cnt = 0;
            for(int j = 1; j <= n; j++) {
                if(is[j]) g[++cnt] = j;
            }
            cout << cnt << "\n";
            for(int p = 1; p <= cnt / d; p++) {
                cout << g[p * d] << " ";
            }
        }
    }
}

```



```

    }
    cout << "\n";
    return;
}
}
cout << "0\n\n";
}
int main() {
    int T;
    cin >> T;
    while(T--) solve();
    return 0;
}

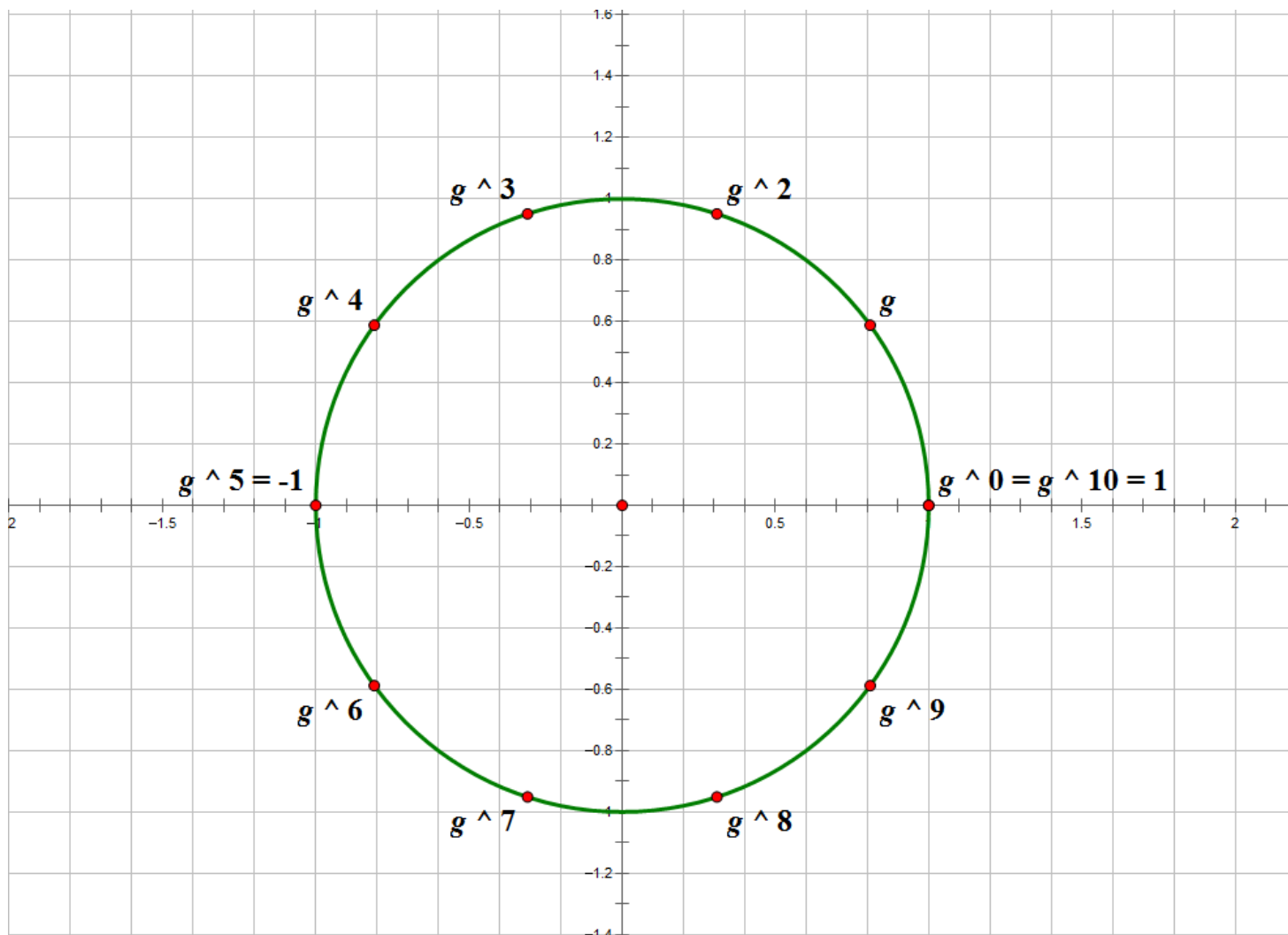
```

补充：因为 $\frac{n}{\varphi(n)}$ 是 $\mathcal{O}(\log \log n)$ 级别，所以 $\frac{n}{\varphi(\varphi(n))} = \mathcal{O}((\log \log n)^2)$ 。如果每次在 $1 \sim m - 1$ 中均匀随机选择一个数，那么期望随机 $\mathcal{O}((\log \log m)^2)$ 次得到一个原根，比 $\mathcal{O}(m^{0.25})$ 更好。

原根和单位根

若 m 有原根 g ，设 $n = \varphi(m)$ ，则 m 的简化剩余系与 n 次单位根 **同构**。

对于任何 $a \perp m$ ，其可以写成 g^k 。任何一个 n 次单位根也可以写成 ω_n^k 。同时 $g^n \equiv 1 \pmod{p}$ 且 $\omega_n^n = 1$ 。因此，我们可以将单位圆应用在 m 的简化剩余系上，为原根提供了一个形象的表示方法，如下图。



图中，单位根的 $0 \sim 9$ 次幂等分单位圆。 $\omega_n^i \times \omega_n^j = \omega_n^{(i+j) \bmod 10}$ 对应 $g^i \times g^j \equiv g^{(i+j) \bmod 10} \pmod{11}$ ，环上运算（两个数相乘）的循环往复（逆时针旋转）体现了“模”的操作。

因此，对于特定模数，我们可以使用原根代替 n 次单位根进行快速傅里叶变换 FFT。当需要使用 2^k 次单位根时，若 $2^k \mid \varphi(p)$ 且 p 有原根，可使用原根代替 2^k 次单位根，即快速数论变换 NTT。常见模数 998244353 的 Euler 函数 $\varphi(998244353) = 2^{22} \times 238$ 。

类比原根和单位根，得到以下性质。

性质 7

设 p 是奇质数或奇质数的幂， g 是 p 的原根，则 $g^{\varphi(p)/2} \equiv -1 \pmod{p}$ 。

证明

设 $g^x \equiv -1 \pmod{p}$ ，则 $x \neq \varphi(p)$ 。如果 $x \neq \frac{\varphi(p)}{2}$ ，那么 $(g^x)^2 = g^{2x} \not\equiv 1 \pmod{p}$ ，和 $g^x \equiv -1 \pmod{p}$ 矛盾。因此 $\log_p -1 = \frac{\varphi(p)}{2}$ 。□

性质 7 在二次剩余部分有用。

*6.3 $(\mathbb{Z}/n\mathbb{Z})^\times$ 的结构

关于记号，见 1.10 小节。

根据 CRT，我们只需要考虑 $(\mathbb{Z}/p^k\mathbb{Z})^\times$ 的结构。

根据原根的定义， $(\mathbb{Z}/p^k\mathbb{Z})^\times$ 是循环群当且仅当 p^k 有原根。对奇质数 p ，已知 p^k 有原根，所以 $(\mathbb{Z}/p^k\mathbb{Z})^\times$ 一定是循环群。实际上，证明 $(\mathbb{Z}/p^k\mathbb{Z})^\times$ 和证明 p^k 有原根等价。笔者在查阅资料时找到了一篇分析 $(\mathbb{Z}/n\mathbb{Z})^\times$ 的结构的论文，其中直接证明 $(\mathbb{Z}/p^k\mathbb{Z})^\times$ 是循环群的推导过程和以上证明 p^k 有原根是几乎一致的。

考虑 $(\mathbb{Z}/2^k\mathbb{Z})^\times$ 的结构。当 $k = 1$ 或 $k = 2$ 时平凡。读者可以先猜一猜结果会是什么样。由有限生成阿贝尔群基本定理， $(\mathbb{Z}/2^k\mathbb{Z})^\times$ 同构于若干循环群的直积，如果有一个元素的阶是 2^{k-2} ，那么 $(\mathbb{Z}/2^k\mathbb{Z})^\times$ 只能是 $Z_2 \times Z_{2^{k-2}}$ ，因为 $|(\mathbb{Z}/2^k\mathbb{Z})^\times| = 2^{k-1}$ 。

事实上，5 就是这样的元素。

引理 1

对 $k \in \mathbb{N}$ ， $5^{2^k} - 1$ 恰含有 $k + 2$ 个质因数 2。

证明

当 $k = 0$ 时命题成立。 $5^{2^{k+1}} - 1 = (5^{2^k} - 1)(5^{2^k} + 1)$ ，前一项恰因子含有 $k + 2$ 个质因数 2，是 4 的倍数，所以后一项因子不是 4 的倍数。因此 $5^{2^{k+1}} - 1$ 恰含有 $k + 3$ 个质因数 2。□

引理 2

对 $k \geq 3$ ， $\delta_{2^k}(5) = 2^{k-2}$ 。

证明

因为阶是 2^{k-1} 的因数但不能等于 2^{k-1} （没有原根），所以 $\delta_{2^k}(5) \mid 2^{k-2}$ 。如果 $5^{2^{k-3}} \equiv 1 \pmod{2^k}$ ，那么 $5^{2^{k-3}} - 1$ 含有至少 k 个质因数 2，和引理 1 矛盾。因此 $\delta_{2^k}(5) = 2^{k-2}$ 。□

现在我们知道 5 是 $Z_{2^{k-2}}$ 这一项的生成元，那么 Z_2 这一项的生成元是什么？找一个阶为 2 的元素，也就是找一个不是 1 但平方是 1 的数。我们知道这样的数有 3 个，-1 是其中之一，那么 -1 是否合法呢？

引理 3

对 $k \geq 2$ 和 $m \in \mathbb{N}$ ， $5^m \not\equiv -1 \pmod{2^k}$ 。

证明

如果成立，那么 $5^m \equiv -1 \pmod{4}$ ，但 $5^m \equiv 1^m \equiv 1 \pmod{4}$ ，矛盾。□

上述推导说明 $(\mathbb{Z}/2^k\mathbb{Z})^\times \cong Z_2 \times Z_{2^{k-2}}$ ，其中生成元分别是 -1 和 5 。于是得到以下结论。

性质 8

对任意 $k \geq 3$ 和 $a \perp 2$ ，存在 $c \in [0, 1]$ 和 $d \in [0, 2^{k-2} - 1]$ 使得 $a \equiv (-1)^c 5^d \pmod{2^k}$ ，且这种表示唯一，即每个 $a \in [0, 2^k - 1]$ 的奇数唯一对应一组 (c, d) 。

性质 8 是求模 2^k 下的 N 次剩余的关键结论。

现在让我们站在更高的视角看看为什么 2^k ($k \geq 3$)， $4p$ ($p \perp 2$) 和 pq ($p, q \perp 2$) 没有原根：由 CRT，对 $n \perp m$ ， $(\mathbb{Z}/nm\mathbb{Z})^\times \cong (\mathbb{Z}/n\mathbb{Z})^\times \times (\mathbb{Z}/m\mathbb{Z})^\times$ ，于是

$$\begin{aligned} (\mathbb{Z}/2^k\mathbb{Z})^\times &\cong Z_2 \times Z_{2^{k-2}} \\ (\mathbb{Z}/4p\mathbb{Z})^\times &\cong (\mathbb{Z}/4\mathbb{Z})^\times \times (\mathbb{Z}/p\mathbb{Z})^\times \cong Z_2 \times Z_{p-1} \\ (\mathbb{Z}/pq\mathbb{Z})^\times &\cong (\mathbb{Z}/p\mathbb{Z})^\times \times (\mathbb{Z}/q\mathbb{Z})^\times \cong Z_{p-1} \times Z_{q-1} \end{aligned}$$

由 3.3 小节，如果 n 和 m 不互质，那么 $Z_n \times Z_m$ 一定不是循环群，所以这些数对应的模意义下的乘法群都不是循环群，自然没有原根。

6.4 例题

P5605 小 A 与两位神仙

前置知识：Pollard-rho（数论 III）。

模数 p 是奇质数的幂，存在原根 g 。

尝试将 x 和 y 写成 g^X 和 g^Y 。根据欧拉定理， $x^a \equiv y \pmod{p}$ 等价于方程 $Xa \equiv Y \pmod{\varphi(p)}$ 。根据 Bezout 定理，后者有解的充要条件是 $\gcd(X, \varphi(p)) \mid Y$ ，这等价于 $\gcd(X, \varphi(p)) \mid \gcd(Y, \varphi(p))$ 。根据阶的性质 $\delta_p(g^k) = \frac{\delta_p(g)}{\gcd(\delta_p(g), k)}$ ，我们发现 $\delta_p(x) = \frac{\varphi(p)}{\gcd(\varphi(p), X)}$ ，因此条件又等价于 $\delta_p(y) \mid \delta_p(x)$ 。

直接求阶即可，需要使用 Pollard-rho 分解 $\varphi(p)$ ，时间复杂度 $\mathcal{O}(n \log^2 p + p^{0.25})$ 。

本题告诉我们的结论：对于奇质数幂 p^α 和 $x, y \perp p$ ， $x^a \equiv y \pmod{p^\alpha}$ 有解当且仅当 $\delta_{p^\alpha}(y) \mid \delta_{p^\alpha}(x)$ 。

*P6730 [WC2020] 猜数游戏

和上一题差不多的思路。

分成 a 与 p 互质和 a 与 p 不互质两种情况考虑。并且将每个数的贡献拆开来考虑，即求出有多少种情况使得 a 对答案有贡献。

当且仅当 S 中不存在一个数能生成 a 时， a 对答案有贡献。因此对每个 a ，答案加上 2^{n-c} ，其中 c 是能生成 a 的数的个数。

当 a 与 p 不互质时，因为 p 是奇质数的幂，所以在不超过 $\log$ 次后 a 的幂变为 0，因此直接考虑每个数能生成哪些数即可得到每个数能被多少个生成。使用哈希表存储，该部分时间复杂度 $n \log p$ 。

当 a 与 p 互质时，直接使用上一题的结论，对每个数求阶后做高维后缀和即可。注意若两个数能互相生成，即阶相等，则需要钦定一个顺序，使得前面的数钦定生成后面的数。

7. N 次剩余问题

N 次剩余问题即求解形如 $x^n \equiv a \pmod{p}$ 的方程。注意未知数 x 在底数上，而离散对数问题的未知数在指数上。

7.1 定义与符号

对 $p \nmid a$ ，若存在整数 x 使得 $x^n \equiv a \pmod{p}$ ，则称 a 是 p 的 n 次剩余。

特别地，对 $p \nmid a$ ，若存在整数 x 使得 $x^2 \equiv a \pmod{p}$ ，则称 a 是 p 的 **二次剩余**，反之称 a 为 p 的 **二次非剩余**。注意，这两个定义基于 a 不是 p 的倍数，也就是说，如果 a 是 p 的倍数，那么 a 既不是 p 的二次剩余，也不是 p 的二次非剩余。

Legendre 符号：记作 $\left(\frac{a}{p}\right)$ 。当 a 是 p 的二次剩余时，该值为 1；当 a 是 p 的二次非剩余时，该值为 -1 ；当 a 是 p 的倍数时，该值为 0。

7.2 二次剩余的分布

奇质数

当 p 是奇质数时，二次剩余非常容易理解。如果 a 是 p 的二次剩余，那么 $a \equiv x^2 \pmod{p}$ 。用原根表示 a 和 x ，设 $a \equiv g^d \pmod{p}$ ， $x \equiv g^k \pmod{p}$ ，那么 $g^d \equiv g^{2k} \pmod{p}$ ，即 $d \equiv 2k \pmod{p-1}$ 。因为 $p-1$ 是偶数，所以 d 必须是偶数，否则无解。当 d 是偶数时， k 有两个解（1.3 小节应用部分），每个 k 对应一个 $x \equiv g^k \pmod{p}$ 。

也就是说，原根的偶数次幂和奇数次幂将 $1 \sim p-1$ 分成两类，偶数次幂对应二次剩余，奇数次幂对应二次非剩余。因此，二次剩余和二次非剩余的数量均为 $\frac{p-1}{2}$ 。这和原根的选择无关，也就是说，任意原根的偶数次幂集合都是相同的。

进一步地，如果 x_0 和 x_1 都是 a 的平方根，那么由平方差公式， $(x_0 + x_1)(x_0 - x_1) \equiv 0 \pmod{p}$ 。因为 p 是质数，所以 $x_0 + x_1 \equiv 0 \pmod{p}$ ，即 x_0 和 x_1 互为相反数。从原根的角度分析， x_0 和 x_1 写成以 g 为底的幂时在指数上相差 $\frac{p-1}{2}$ ，而 $g^{(p-1)/2} \equiv -1 \pmod{p}$ 。另一种角度是如果 $x^2 \equiv a \pmod{p}$ ，那么 $(-x)^2 \equiv a \pmod{p}$ 。

类比：正数（二次剩余）的平方根有两个且互为相反数，负数（二次非剩余）没有平方根。

奇质数幂

将奇质数的结论稍微推广即可，读者可以先自行尝试推导结论。

p^k 有原根 g ，所以对于 $a \perp p$ ，依然可以用奇质数的分析方法得到 $\log_g a$ 是偶数时是二次剩余，是奇数时是二次非剩余。同样地，其平方根有两个且互为相反数。

设非零整数 $a = p^c a'$ 。如果 a 是二次剩余，那么存在 $x = p^d x'$ 使得 $a \equiv x^2 \pmod{p^k}$ 。显然 $c = 2d$ 必须是偶数。此外，还要看 a' 是否为 p^{k-c} 的二次剩余。如果是，设 x' 是 $a' \equiv (x')^2 \pmod{p^{k-c}}$ 的解，那么 $p^{c/2} x'$ 是原方程的解，即 a 是二次剩余。否则 a 是二次非剩余。

以上推导可以推广到 N 次剩余的情况。实际上，这是求解 N 次剩余的一部分。

2 的幂

设 $p = 2^k$ 。

首先考虑 a 是奇数的情况。注意到任何奇数的平方模 8 余 1，所以 $a \equiv x^2 \pmod{p}$ 且 x 是奇数的必要条件为 $a \equiv 1 \pmod{8}$ 。那么这个条件是否充分呢？

假设 y 是方程的解，那么 $(x - y)(x + y) \equiv 0 \pmod{p}$ 。因为 y 是奇数，所以 $2y$ 不是 4 的倍数。因此， $x + y$ 和 $x - y$ 至少有一个不是 4 的倍数。如果 $x + y$ 不是 4 的倍数，那么 $x - y$ 必须是 2^{k-1} 的倍数，即此时 x 恰有两个解。类似地，如果 $x - y$ 不是 4 的倍数，那么此时 x 同样恰有两个解。综合两种情况，如果 $a \equiv x^2 \pmod{p}$ 有解，那么至多有 4 个解。这个分析和扩展 Wilson 定理非常类似。

一共有 2^{k-1} 个奇数，且至多有 2^{k-3} 个奇数的平方，但一个平方数最多有 4 个根，那么唯一的情况是所有模 8 余 1 的数都是二次剩余，且它们的平方根都有 4 个。综上，如果 a 是奇数，那么 a 是二次剩余当且仅当 $a \equiv 1 \pmod{8}$ 。

对于 a 是偶数的情况，和奇质数幂的推导类似。 a 是二次剩余当且仅当 $a = 4^k a'$ ，其中 a' 是奇二次剩余。

任意合数

根据 CRT 拆成模质数幂。显然地，对 $p \nmid a$ ， a 是 p 的二次剩余当且仅当对任意拆出的质数幂 q^k ， a 不是 q^k 的二次非剩余。注意并非 a 是 q^k 的二次剩余，区别在于 a 可以是 q^k 的倍数。

7.3 二次剩余的判定：Euler 准则

7.2 小节给出了 2^k 的二次剩余的判定准则，以及其它情况是如何归约到判定奇质数的二次剩余。因此，本节只关注奇质数的情况。

给定奇质数 p ，则 p 有原根 g 。

对于整数 a ，我们希望知道 a 在以 g 为底时的指数是不是偶数。那么有没有什么办法不用求出指数就能确定它的奇偶性呢？提示：利用阶的性质 7。

注意到偶数的 $\frac{p-1}{2}$ 倍是 $p-1$ 的倍数，但奇数的 $\frac{p-1}{2}$ 倍不是 $p-1$ 的倍数。因此，如果 k 是偶数，那么 $g^{k(p-1)/2} \equiv 1 \pmod{p}$ ；如果 k 是奇数，那么根据阶的性质 7， $g^{k(p-1)/2} \equiv -1 \pmod{p}$ 。而 $g^{k(p-1)/2}$ 即 $a^{(p-1)/2}$ 。

Euler 准则

设 p 是奇质数。

$$a^{\frac{p-1}{2}} \equiv \left(\frac{a}{p}\right) \pmod{p}$$

简单地说， a 是二次剩余还是二次非剩余等价于 $a^{\frac{p-1}{2}} \pmod{p}$ ，等价于 a 能否表示成 p 的一个原根的 **偶数** 次幂。读者应仔细体会其中的等价关系。

Euler 准则同时告诉我们 Legendre 符号关于 a 是完全积性函数。即对于任意 a, b ,

$$\left(\frac{a}{p}\right) \left(\frac{b}{p}\right) = \left(\frac{ab}{p}\right)$$

笔者的理解：由 Fermat 小定理，对奇素数 p ， $a^{p-1} \equiv 1 \pmod{p}$ 。指数除以 2 相当于开平方根，此时 $1 \sim p-1$ 会因为自身性质的不同而分化成两类，一类是二次剩余 $a^{(p-1)/2} \equiv 1 \pmod{p}$ ，另一类是二次非剩余 $a^{(p-1)/2} \equiv -1 \pmod{p}$ 。这种分化，用单位根的知识来解释，就是对于 $p-1$ 次单位根 ω_{p-1}^k ，当 k 为偶数时 $(\omega_{p-1}^k)^{(p-1)/2} = 1$ ，反之为 -1 。原根和单位根的联系恰好解释了 $a \equiv g^k \pmod{p}$ ， k 的奇偶性对 a 是否是二次剩余的影响。

7.4 模意义下开平方根

考虑求解 $a \equiv x^2 \pmod{p}$ ，以下只考虑 a 是 p 的二次剩余的情况。

7.4.1 模质数下开平方根

设 p 是奇质数（模 2 开平方根是平凡的），解方程 $a \equiv x^2 \pmod{p}$ ($a \neq 0$)，则 x 即模 p 下的 $\sqrt{x}$ 。

由 7.2 小节的分析，方程恰有两个解，只需求出任意一个解，通过取相反数得到另一个。

使用 BSGS，显然存在根号时间的算法：先求出 $\log_g a$ 。因为 a 是二次剩余，所以 $\log_g a$ 是偶数，则 $g^{\log_g(a)/2}$ 即为所求。

特殊情况的解法：由 Euler 准则， $a^{(p-1)/2} \equiv 1 \pmod{p}$ 。特别地，如果 $\frac{p-1}{2}$ 是奇数，那么 $a^{(p+1)/2} \equiv a \pmod{p}$ 且 $\frac{p+1}{2}$ 是偶数。令 $x = a^{(p+1)/4}$ 即可。

模意义下扩域

设 ω 是二次非剩余， $i^2 \equiv \omega \pmod{p}$ 。这样的 $i = \sqrt{\omega}$ 在模 p 下的整数域中不存在。就像 $\sqrt{-1}$ 在实数域上不存在使得数学家们定义复数域 $\{a + bi \mid a, b \in \mathbb{R}\}$ 一样，我们定义数域 $\mathbb{Z}_p[i] = \{a + bi \mid a, b \in [0, p-1]\}$ ，可以理解为模 p 下以 $\sqrt{\omega}$ 为虚数单位的复数。

该数域满足很多常见的代数性质，如乘法结合律，乘法分配律等，在此不一一证明。其上的运算可以看成模 p 下的多项式的运算，只不过 $x^2 = \omega$ 。

这个技巧常见于模意义下的二次非剩余强制开方。例如 5 在模 $10^9 + 7$ 下没有二次剩余，所以，在计算斐波那契数列模 $10^9 + 7$ 时，需要将一个数表示为 $a + b\sqrt{5}$ 。

由 Euler 准则， $i^{p-1} \equiv \omega^{(p-1)/2} \equiv -1 \pmod{p}$ ，所以 $i^p \equiv -i \pmod{p}$ 。

Cipolla

我们希望找到一个数的偶数次幂等于 a 。考虑 Lucas 定理的引理，

$$(b+x)^{p+1} \equiv (b^p + x^p)(b+x) \pmod{p}$$

令 b 是模 p 下的整数， x 是二次非剩余 ω 开根，记为 i ，则

$$(b+i)^{p+1} \equiv (b-i)(b+i) \equiv b^2 - \omega \pmod{p}$$

也就是说，如果能找到正整数 b 使得 $\omega = b^2 - a$ 是二次非剩余，那么 $(b+i)^{(p+1)/2}$ 就是 a 在 $\mathbb{Z}_p[i]$ 上的二次剩余。

- $(x+y)^p \equiv x^p + y^p \pmod{p}$ 在 $\mathbb{Z}_p[i]$ 上的正确性：使用二项式定理展开后，考虑到对任意 $a+bi \in \mathbb{Z}_p[i]$ ， $p(a+bi) \equiv 0+0i \equiv 0 \pmod{p}$ 。

- 假如存在 $(c + di)^2 \equiv a \pmod{p}$ ，对比两侧虚部得到 $2cd \equiv 0 \pmod{p}$ 。如果 $d \neq 0$ ，那么 $c = 0$ ，即 $d^2 \omega \equiv a \pmod{p}$ 。因为 a 是二次剩余，所以 ω 是二次剩余（Legendre 符号的完全积性），矛盾。因此 $d = 0$ ，即 a 在 $\mathbb{Z}_p[i]$ 上的二次剩余也一定是在 $\mathbb{Z}_p$ 上的二次剩余（ p 的二次剩余），也即 $(b + i)^{(p+1)/2}$ 的虚部为 0。
- 考虑 $b^2 - a$ 是二次剩余的 b 的数量，即存在 $x \in [1, p-1]$ 使得 $b^2 - a \equiv x^2 \pmod{p}$ 即 $(b-x)(b+x) \equiv a \pmod{p}$ 。固定 $b+x \in [1, p-1]$ 得到 $b-x$ ，可以唯一求出一组解 (b, x) ，而固定 b 之后对应的 x 恰有两个（开根有两个解）。去掉 $x = 0$ 的两种情况（ $b^2 \equiv a \pmod{p}$ ），可知 b 的数量为 $\frac{(p-1)-2}{2}$ 。因此，使得 $b^2 - a$ 是二次非剩余的 b 的数量为 $p - \frac{p-3}{2} - 2 = \frac{p-1}{2}$ 。这说明期望随机两次 b 即可得到 ω 。

综上，Cipolla 算法的时间复杂度为 $\mathcal{O}(\log p)$ 。模板题 [P5491](#) 代码。

```

#include <bits/stdc++.h>
using namespace std;
mt19937 rnd(1064);
int T, n, p, b, w;
struct comp {
    int a, b;
    comp operator + (comp c) {
        return {(a + c.a) % p, (b + c.b) % p};
    }
    comp operator * (comp c) {
        return {(111 * a * c.a + 111 * b * c.b % p * w) % p, (111 * a * c.b + 111 * b * c.a) % p};
    }
};
int ksm(int a, int b) {
    int s = 1;
    while(b) {
        if(b & 1) s = 111 * s * a % p;
        a = 111 * a * a % p, b >>= 1;
    }
    return s;
}
comp ksm(comp a, int b) {
    comp s = {1, 0};
    while(b) {
        if(b & 1) s = s * a;
        a = a * a, b >>= 1;
    }
    return s;
}
void solve() {
    cin >> n >> p;
    if(!n) return puts("0"), void();
    if(ksm(n, p - 1 >> 1) == p - 1) return puts("Hola!"), void();
    do b = rnd() % p; while(ksm(w = (111 * b * b - n + p) % p, p - 1 >> 1) != p - 1);
    int x = ksm({b, 1}, p + 1 >> 1).a;
    cout << min(x, p - x) << " " << max(x, p - x) << "\n";
}
int main() {
    cin >> T;
    while(T--) solve();
    return 0;
}

```

除了 Cipolla 以外，还有能够简单扩展到模质数幂的 Tonelli-Shanks 算法，此处不做介绍，详见参考资料。

7.4.2 任意模数下开平方根

奇质数幂

设模数为 p^k , $a = p^c a'$ 。特别地, 由 7.2 小节, a 是二次剩余当且仅当 c 是偶数且 a' 是二次剩余, 可以通过 $a' \equiv x^2 \pmod{p^{k-c}}$ 的解乘以 $p^{c/2}$ 得到 $a \equiv x^2 \pmod{p^k}$ 的解, 所以接下来只考虑 $a \perp p^k$ 即 $a \perp p$ 的情况。

处理模质数幂的第一步一般是先把答案在模质数下求出来。设 x 是 a 在模 p 下的平方根, 则 $x^2 - a \equiv 0 \pmod{p}$ 。设 $\sqrt{a}$ 是 a 在模 p^k 下的平方根。将同余式两侧 k 次方, 得

$$(x^2 - a)^k \equiv (x + \sqrt{a})^k (x - \sqrt{a})^k \equiv 0 \pmod{p^k}$$

因为 $a \perp p$, 所以 $\sqrt{a}$ 不是 p 的倍数。于是 $x + \sqrt{a}$ 和 $x - \sqrt{a}$ 不可能都是 p 的倍数, 那么一定恰有一个是 p 的倍数。

把 $\sqrt{a}$ 看成未知数待定, 展开 $(x + \sqrt{a})^k$ 得到 $u + v\sqrt{a}$, 那么展开 $(x - \sqrt{a})^k$ 的结果一定是 $u - v\sqrt{a}$ 。于是

$$(u + v\sqrt{a})(u - v\sqrt{a}) \equiv u^2 - v^2 a \equiv 0 \pmod{p^k}$$

又因为 $u + v\sqrt{a}$ 和 $u - v\sqrt{a}$ 恰有一个是 p 的倍数, 所以 $v\sqrt{a}$ 一定不是 p 的倍数, 即 $v \perp p$, 于是 $v \perp p^k$ 。根据上式, $u^2 \equiv v^2 a \pmod{p^k}$, 令 $y \equiv uv^{-1} \pmod{p^k}$, 则 $y^2 \equiv u^2 v^{-2} \equiv a \pmod{p^k}$ 。

求出一个平方根后, 取相反数得到另一个, 即得所有平方根。

时间复杂度 $\mathcal{O}(\log p)$ 。

2 的幂

类似地, 我们只考虑 a 是奇数的情况。

如果 $a \equiv x^2 \pmod{2^{k+1}}$, 那么 $a \equiv x^2 \pmod{2^k}$ 。因此, 如果 x 是 a 在模 2^{k+1} 下的平方根, 那么它一定是 a 在模 2^k 下的平方根。

因此, 考虑 a 在模 2^k 下的所有平方根 $x_1, \dots, x_t$, 只需检查 x_i 和 $x_i + 2^k$ 是否为 a 在模 2^{k+1} 下的平方根即可。由 7.2 小节, $t \leq 4$ 。

显然地, 该算法可以求出所有平方根。如果认为平方根的数量是常数, 则时间复杂度 $\mathcal{O}(\log p)$ 。

也可以使用类似开 N 次方根的算法, 时间复杂度较劣。见 7.5 小节。

任意模数

使用 CRT 合并即可。如果要求出所有平方根，当 a 是二次剩余时 $x^2 \equiv a \pmod{p^k}$ 的所有解的求法已经在上文介绍了，还需考虑当 $x^2 \equiv 0 \pmod{p^k}$ 的所有解。这个问题的答案是显然的： $x = p^{k'} x'$ ，其中 $2k' \geq k$ 。

*7.5 任意模数开 N 次根

在掌握了足够多二次剩余相关性质和开平方根的算法之后，我们考虑解决更一般 N 次剩余问题： $x^n \equiv a \pmod{p}$ 。其中， $a = 0$ 的情况可类似开平方根推导，不再赘述。

奇质数

设 g 是 p 的原根， $a \perp p$ 。使用 BSGS 求出 $d = \log_g a$ 。如果 $x \equiv g^k$ 是 n 次根，那么 $nk \equiv d \pmod{p-1}$ ，解这个同余方程即可。

奇质数幂

设模数为 p^k ， $a = p^c a'$ 。如果 $x = p^d x'$ 是 n 次根，那么 $c = nd$ 是 n 的倍数。除掉这部分之后解 $(x')^n \equiv a' \pmod{p^{k-c}}$ ，类似模奇质数用原根 + BSGS 即可。

*2 的幂

类似地，容易简化至 $a \perp 2$ 且 $k \geq 3$ 的情况。

开平方根时的递推算法在这里不适用了，因为递推过程中解的数量可能非常大，但最终它们都无法成为 n 次根。这导致就算保证了最终答案的数量，递推算法的复杂度依然无法保证，见 [这个帖子](#)。

根据 6.3 小节的结论，所有 $a \perp 2$ 可以写成唯一的 $(-1)^c 5^d$ 。我们使用 BSGS 求出这样的表示：如果 $\log_5 a$ 存在，那么 $c = 0$ ， $d = \log_5 a$ 。否则 $c = 1$ ， $d = \log_5(-a)$ 。如果 $x = (-1)^{c'} 5^{d'}$ 是 n 次根，那么有同余方程组

$$\begin{cases} nc' \equiv c \pmod{2} \\ nd' \equiv d \pmod{2^{k-2}} \end{cases}$$

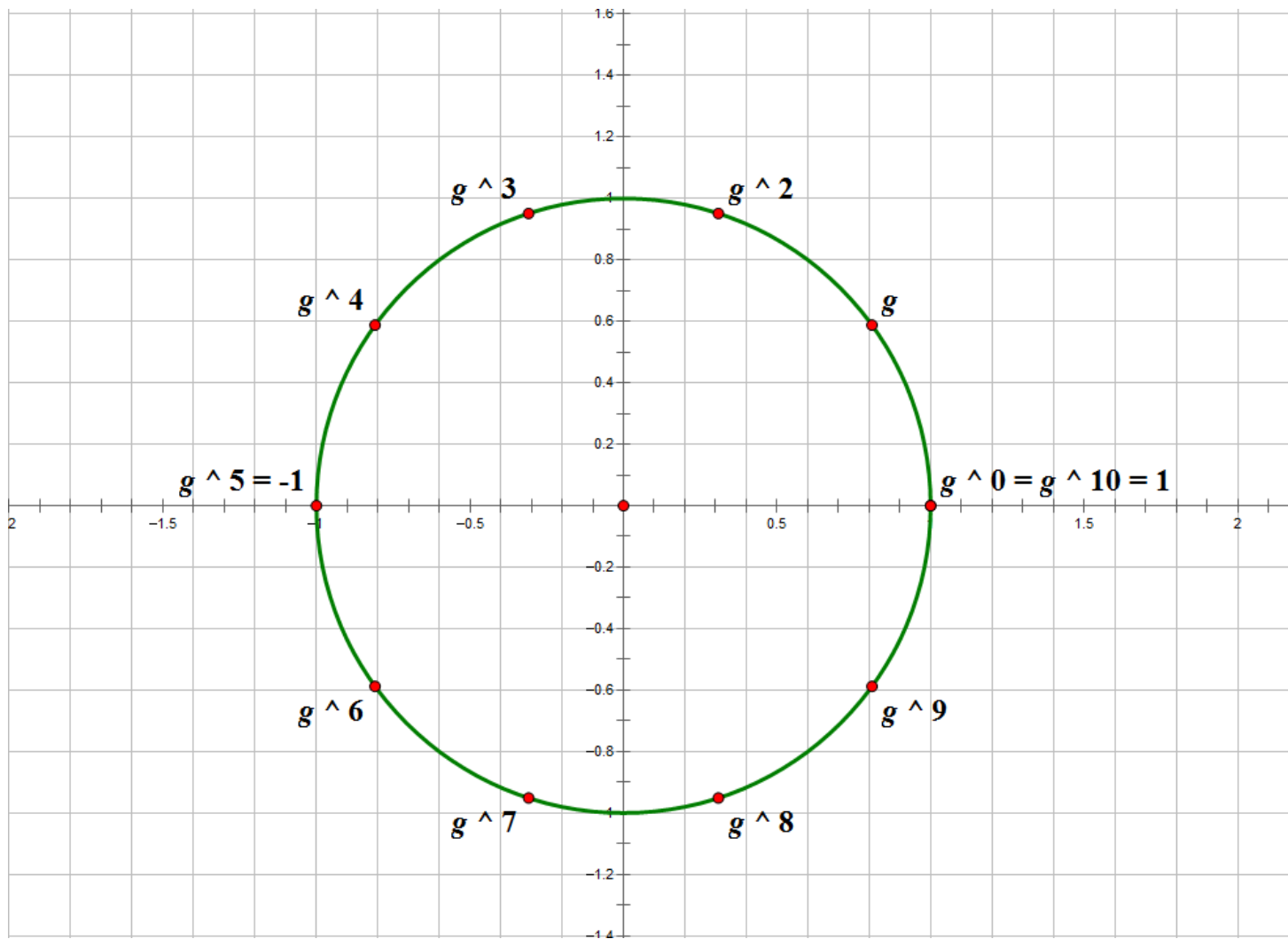
两个方程独立，分别求解后再合并即可。

任意模数

使用 CRT 合并即可。如果认为平方根的数量是常数，则时间复杂度 $\mathcal{O}(\sqrt{p})$ 。

7.6 总结

N 次剩余相关的知识比较难，为了更好地理解这些结论和算法，当模数有原根时，抓住原根等价于单位根的性质，想象一个环是很有帮助的。每个数编号为其以 g 为底的离散对数，两个数相乘相当于环上的编号相加。一个数的编号给出了它的很多性质，编号的奇偶性决定了其能否表示成一个数的平方，也引出了 Euler 准则。这对理解高次剩余同样有帮助。是时候再次贴出这张图了。



接下来我们介绍了模意义下开根，这个问题有三个维度：

- $a = 0 / a$ 是剩余。
- 模质数 / 奇质数幂 / 2 的幂 / 任意合数。
- $n = 2 / n \geq 2$ 。

对于不同的情况有复杂度和复杂程度不同的解法。 $n = 2$ 时可以做到 $\mathcal{O}(\log p)$ ，而 $n \geq 2$ 时本文仅介绍了 $\mathcal{O}(\sqrt{p})$ 。

在模数没有原根的时候，我们以 6.3 小节 $(\mathbb{Z}/n\mathbb{Z})^\times$ 的结构为基础，同样能够把开方操作转化为指数上简单的同余方程。一个例子是模 2 的幂下开 N 次根。

限于篇幅和OI方面的应用，还有很多内容没有介绍，如二次互反律。感兴趣的读者可以查阅资料自行学习。

7.7 例题

*P6610 [Code+#7] 同余方程

根据 $\mu(p) \neq 0$ 不难想到使用 CRT 将问题拆成模奇质数。每个奇质数的解的个数的积即为所求。设 p 是奇质数。

问题转化为求 x 能被拆成多少组 a, b 的和，使得 a, b 均为二次剩余或 0。其中，每个二次剩余的贡献系数是 2（一个二次剩余可表示为两个不同的数的平方），0 的贡献是 1，每组 (a, b) 的贡献即 a 和 b 分别的贡献系数之积。

我们需要数学公式而非判断式来描述答案，因为判断式不可化简。

二次剩余对应 2，0 对应 1，二次非剩余对应 0，这使得我们想到 Legendre 符号：注意到 $\left(\frac{a}{p}\right) + 1$ 等价于使得 $x^2 \equiv a \pmod{p}$ 的 x 的个数，因此答案为

$$\sum_{a+b \equiv x} \left(\left(\frac{a}{p} \right) + 1 \right) \left(\left(\frac{b}{p} \right) + 1 \right)$$

$1 \sim p-1$ 的二次剩余和二次非剩余个数相等，所以 $\sum_{i=0}^{p-1} \left(\frac{i}{p} \right) = 0$ 。再根据 $b \equiv x - a \pmod{p}$ 和 Legendre 符号的完全积性，上式化简为

$$p + \sum_{a=0}^{p-1} \left(\frac{ax - a^2}{p} \right)$$

接下来是一步巧妙转化。考虑到给 $ax - a^2$ 除以 a^2 并不影响二次剩余性，因此答案即

$$p + \sum_{a=1}^{p-1} \left(\frac{\frac{x}{a} - 1}{p} \right)$$

显然，对于 $x \neq 0$ 且 $a \in [1, p-1]$ ， $\frac{x}{a}$ 取遍 $[1, p-1]$ ，即 $\frac{x}{a} - 1$ 取遍 $[0, p-2]$ 。故 $x \neq 0$ 时，答案等于

$$p - \left(\frac{-1}{p} \right)$$

根据 Euler 准则，上式即 $p - (-1)^{\frac{p-1}{2}}$ 。

对于 $x = 0$ ，答案 $p + \sum_{a=1}^{p-1} \left(\frac{-1}{p}\right)$ 即 $p + (p-1)(-1)^{\frac{p-1}{2}}$ 。

公式足够简洁，可以 $\mathcal{O}(1)$ 计算。时间复杂度 $\mathcal{O}(p + n\omega(p))$ 。[代码](#)。

*P8457 「SWTR-8」 幂塔方程

[题解](#)。

CHANGE LOG

- 2021.12.6：重构文章，删去线性筛部分，修改部分表述。
- 2022.3.15：重构文章。
- 2022.3.24：新增 Wilson 定理，素数在阶乘和组合数中的幂次，阶与原根，二次剩余和 Lucas 定理。
- 2022.6.12：勘误，修改表述。
- 2022.6.22：将数论分块移出本文。
- 2024.2.22：增加在线 $\mathcal{O}(1)$ 逆元。
- 2025.1.22：重构文章，移除欧拉函数。
- 2025.1.27：补充任意模数开平方根，开 N 次方根和 $(\mathbb{Z}/n\mathbb{Z})^\times$ 的结构性质。

参考资料

第一章

- [在线 \$\Theta\(1\)\$ 逆元](#) —— zhoukangyang。

第二章

- [关于 exgcd 求得特解的数值范围](#) —— ycx060617。

第六章

- [原根 - OI Wiki](#)。
- The Structure of $(\mathbb{Z}/n\mathbb{Z})^\times$ —— R. C. Daileida。 [PDF 链接](#)。

第七章

- [Quadratic residue](#) —— Wikipedia。
- [Cipolla's algorithm](#) —— Wikipedia。
- [二次剩余](#) —— Aleph_Drawer（关于 $b^2 - a$ 是二次非剩余的 b 的数量）。
- [题解 P5491](#) —— cyffff。
- [Tonelli-Shanks algorithm](#) —— Wikipedia。

全文：

- [数论基础学习笔记](#) —— ycx060617。
- [数论](#) —— OI Wiki。
- [抽象代数课程笔记 I](#) —— [群论](#) —— Alex_Wei。
- [抽象代数课程笔记 II](#) —— [环论](#) —— Alex_Wei。